LSTM & Transformers

Study Notes – Working Draft

Stefan Eder

Based on Hochreiter & Adler, lecture notes, WS 2026

These notes are here to make sequence models click, without having to fight your way through a long textbook. We start with the idea, work through the equations, and use small examples when things get tricky. Chapters 0–6, 9, and 10 are written; chapters 7, 8, and 11 are still an outline. Figure placeholders reserve room for diagrams we will add next. The finished notes should stay within about 120 pages.

Corrections and suggestions are welcome. Please open an issue or pull request at **github.com/steveTgit/AI_Master**.

Good study materials should be shared, not gatekept. If this helped you, take it as a reason to help the next colleague you see struggling with something.

September 21, 2026

Contents

0 Notation 1
- 0.1 Scalars, Vectors, and Matrices 1
- 0.2 Sizes and Dimensions 1
- 0.3 Objects and Indices 1
- 0.4 Multiple Layers 2
- 0.5 Stacked Data and Labels 2
- 0.6 Common Transformations 2
- 0.7 Functions 2
- 0.8 Generalized Functions 3
- 0.9 Loss, Risk, and Regularization 3
- 0.10 Derivatives and Matrix Layout 3
- 0.11 Distributions 3

1 Simple Recurrent Networks 4
- 1.1 Jordan Network 4
  - 1.1.1 Teacher Forcing 5
- 1.2 Elman Network 5
- 1.3 Fully Recurrent Network 5
- 1.4 ARMA 6
- 1.5 NARX Recurrent Neural Networks 7
- 1.6 Time-Delay Neural Networks 7

2 Learning Algorithms for RNNs 8
- 2.1 Empirical Risk Minimization via Gradient Descent 8
- 2.2 Backpropagation Through Time 8
  - 2.2.1 BPTT for the Fully Recurrent Network 9
- 2.3 Regularization 11
  - 2.3.1 Early Stopping 11
  - 2.3.2 Weight Decay 11
  - 2.3.3 Dropout 11
- 2.4 Other RNN Learning Algorithms 12
  - 2.4.1 Truncated Backpropagation Through Time 12
  - 2.4.2 Real-Time Recurrent Learning 12
  - 2.4.3 Schmidhuber's Approach 13
- 2.5 The Vanishing Gradient Problem 13
  - 2.5.1 Long-Term Dependencies 13
  - 2.5.2 Vanishing and Exploding Gradients 13
  - 2.5.3 Focused Backpropagation 14

3 Long Short-Term Memory 16
- 3.1 Backpropagation for LSTM 17
  - 3.1.1 The LSTM Learning Method 18
- 3.2 Forget Gate 18

3.3 Tricks of the Trade . . . 20
3.4 Dropout for LSTM . . . 21
3.5 Application Examples . . . 21
3.5.1 Sequence-to-Sequence Learning with LSTM . . . 21
3.5.2 Generating Sequences with LSTMs . . . 22
3.5.3 Image Captioning . . . 23
3.5.4 Learning to Learn Using LSTM . . . 23
3.5.5 Rainfall-Runoff Modelling . . . 23
3.5.6 Talking Heads . . . 24
3.6 LSTM Variants . . . 24
3.6.1 Peephole Connections . . . 24
3.6.2 Bidirectional LSTM . . . 24
3.6.3 Multidimensional LSTM . . . 24
3.6.4 Pyramid LSTM . . . 25
3.6.5 Stacked LSTM . . . 25
3.6.6 Grid LSTM . . . 25
3.6.7 Convolutional LSTM . . . 25
3.7 Advanced LSTM Architectures . . . 25
3.7.1 Neural Turing Machine . . . 25
3.7.2 Differentiable Neural Computer . . . 26
3.7.3 Pointer Network . . . 26
3.8 Gated Recurrent Unit . . . 26
3.8.1 Backpropagation for GRU . . . 27
3.8.2 Usage . . . 27

4 Attention . . . 28
4.1 Introduction . . . 28
4.1.1 Spatial Attention . . . 28
4.1.2 Gates Introduced Temporal Attention . . . 29
4.1.3 Temporal Attention in Artificial Neural Networks . . . 29
4.2 Attention in Sequence-to-Sequence Models . . . 30
4.2.1 Additive Attention . . . 30
4.2.2 Multiplicative Attention and Local Attention . . . 31
4.2.3 Self-Attention . . . 32
4.2.4 Sentence Embedding via Self-Attention . . . 32
4.3 Key-Value Attention . . . 33

5 Transformer . . . 34
5.1 Architecture . . . 34
5.2 Detailed Explanation of the Transformer . . . 35
5.3 Bidirectional Encoder Representations from Transformers (BERT) . . . 42
5.4 LLM Architectures . . . 43
5.4.1 GPT-2 . . . 43
5.4.2 DeepSeek V3/R1 . . . 43
5.4.3 OLMo 2 . . . 44
5.4.4 Gemma 3 . . . 44
5.4.5 Mistral Small 3.1 . . . 45
5.4.6 Llama 4 . . . 45
5.4.7 Qwen3 . . . 45
5.4.8 SmolLM3 . . . 46

5.4.9 Kimi 2 . . . 46
5.4.10 OpenAI GPT OSS . . . 46

6 Linear Attention: Outer Product Memories . . . 47
6.1 Outer-Product Memory and Retrieval . . . 47
6.2 Recurrent Formulation and Complexity . . . 49

7 MAMBA . . . 52
7.1 Mamba Introduction . . . 52
7.1.1 Core Idea . . . 52
7.1.2 State Space Models . . . 52
7.1.3 Selective State Spaces . . . 52
7.1.4 Computational Complexity . . . 52
7.1.5 Comparison with Transformers . . . 52
7.1.6 Relation to RNNs . . . 52
7.1.7 Limitations . . . 53
7.1.8 Applications . . . 53
7.2 Mamba Architecture . . . 53
7.3 Mamba Experimental Evaluation . . . 53
7.3.1 Mamba Scaling Laws . . . 53
7.3.2 Mamba Zero-Shot Performance . . . 53

8 xLSTM . . . 54
8.1 xLSTM Introduction . . . 54
8.1.1 Background: Classical LSTM . . . 54
8.1.2 Motivation for xLSTM . . . 54
8.1.3 Core Ideas of xLSTM . . . 54
8.1.4 xLSTM Architecture Variants . . . 54
8.1.5 Computational Properties . . . 55
8.1.6 Comparison with Transformers . . . 55
8.1.7 Comparison with Mamba . . . 55
8.1.8 Strengths of xLSTM . . . 55
8.1.9 Limitations . . . 55
8.1.10 Broader Significance . . . 55
8.2 xLSTM: Mathematical and Technical Details . . . 55
8.2.1 sLSTM: scalarLSTM . . . 55
8.2.2 Stabilizer State . . . 55
8.2.3 mLSTM: matrixLSTM . . . 55
8.2.4 mLSTM vs. sLSTM . . . 55
8.2.5 xLSTM Architecture . . . 55
8.3 xLSTM Experiments . . . 56
8.3.1 Formal Languages and State Tracking . . . 56
8.3.2 Memory Capacity . . . 56
8.3.3 Language and LLM . . . 56
8.3.4 Length Extrapolation . . . 56
8.3.5 Scaling Laws . . . 56
8.3.6 Speed Comparisons . . . 56
8.3.7 Vision-LSTM (ViL) . . . 56
8.3.8 Bio-xLSTM . . . 56
8.3.9 Robotic xLSTM: LRAM . . . 56
8.3.10 xLSTM Distillation . . . 56

9 TiRex **57**
9.1 Introduction to TiRex 57
9.1.1 Motivation 57
9.1.2 Core Idea 57
9.1.3 State Tracking with sLSTM 58
9.1.4 Foundation Model Perspective 59
9.1.5 Computational Advantages 59
9.1.6 Comparison with Transformer Forecasting Models 59
9.1.7 Why State Tracking Matters 60
9.1.8 Applications 60
9.1.9 Broader Significance 60
9.2 Evaluations and Benchmarks for TiRex 60
9.3 Applications of TiRex 62

10 Attractor Networks **63**
10.1 Backpropagation for attractor networks with continuous time 63
10.2 Hopfield networks 65
10.3 Boltzmann machine 66
10.3.1 Learning in Boltzmann Machines 67
10.3.2 Restricted Boltzmann Machines 68
10.3.3 Deep Boltzmann Machines 69

11 Hidden Markov Models **70**
11.1 Hidden Markov Models in Bioinformatics 70
11.2 Hidden Markov Model Basics 70
11.3 Expectation Maximization for HMM: Baum-Welch Algorithm 70
11.4 Viterbi Algorithm 70
11.5 Input Output Hidden Markov Models 70
11.6 Factorial Hidden Markov Models 70
11.7 Memory Input Output Factorial Hidden Markov Models 71
11.8 Tricks of the Trade 71
11.9 Profile Hidden Markov Models 71

0

Notation

Before we get into the models, let's make the notation less annoying. Samples, time steps, features, and layers all need their own indices. Keep those apart and check matrix shapes as you go: that already catches a surprising number of mistakes. You can come back to this chapter whenever a symbol gets in the way.

0.1 Scalars, Vectors, and Matrices

A scalar x is one number, a bold lower-case symbol $\mathbf{x}$ is a column vector, and a bold upper-case symbol $\mathbf{X}$ is a matrix. Thus $\mathbf{x} \in \mathbb{R}^D$ has shape $D \times 1$, whereas $\mathbf{x}^\mathsf{T}$ has shape $1 \times D$. The expressions $\mathbf{x}^\mathsf{T}\mathbf{x}$ and $\mathbf{x}\mathbf{x}^\mathsf{T}$ are very different: the first is a scalar, the second a $D \times D$ matrix.

0.2 Sizes and Dimensions

The dimensions below recur throughout the summary. Local dimensions are always stated when their meaning changes.

Symbol	Meaning	Typical index
N, M	Training samples; validation or test samples	n, m
D, K	Input features; output components	d, k
I	Hidden units in the recurrent chapters	i, j
L	Number of hidden layers in the layer convention below	l
T, B	Sequence length; mini-batch size	t, n

Letters such as I and J can mean different layer widths in different contexts. The stated matrix shape is what matters; the letter itself is just a label.

0.3 Objects and Indices

An input is $\mathbf{x}^{(n)} \in \mathbb{R}^D$, its label is $\mathbf{y}^{(n)} \in \mathbb{R}^K$, and the model prediction is $\widehat{\mathbf{y}}^{(n)}$. A scalar target uses $y^{(n)}$ instead. Parenthesized superscripts identify samples, not powers. The scalar $x_d^{(n)}(t)$ is feature d of sample n at time t; $\mathbf{x}^{(n)}(1{:}T)$ denotes the whole sequence.

A neural layer maps pre-activations $\mathbf{s}$ to activations $\mathbf{a} = f(\mathbf{s})$. All trainable parameters form $\boldsymbol{\theta}$ (also often called $\mathbf{w}$); $\mathbf{W}$ denotes one weight matrix.

0.4 Multiple Layers

Square brackets distinguish depth from sample and time indices. With $n_h^{[l]}$ units in hidden layer l, let $n_h^{[0]} = D$ and $n_h^{[L+1]} = K$. We store weights with input units as rows and output units as columns:

$$\mathbf{W}^{[l]} \in \mathbb{R}^{n_h^{[l-1]} \times n_h^{[l]}}, \qquad \mathbf{s}^{[l]} = (\mathbf{W}^{[l]})^\mathsf{T}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}, \qquad \mathbf{a}^{[l]} = f(\mathbf{s}^{[l]}).$$

The bias has one component per receiving unit. A convention that writes $\mathbf{Wx}$ instead stores the transposed matrix.

0.5 Stacked Data and Labels

A conventional dataset table has one sample per row: $\mathbf{X}_{\text{data}} \in \mathbb{R}^{N \times D}$ and $\mathbf{Y}_{\text{data}} \in \mathbb{R}^{N \times K}$. Scalar labels form a vector in $\mathbb{R}^N$. In recurrent mini-batches, however, we place simultaneous samples in *columns*:

$$\mathbf{X}(t) = \left[\mathbf{x}^{(1)}(t) \;\cdots\; \mathbf{x}^{(B)}(t)\right] \in \mathbb{R}^{D \times B}.$$

This makes the single-sample and batch equations look the same. Time remains a separate axis: one such matrix is needed for each t. Rows versus columns is a storage choice, not a different model. Just keep the choice consistent.

0.6 Common Transformations

An affine map first combines features and adds a bias; the activation then acts on the resulting pre-activations. For a single neuron this is $a = f(\mathbf{w}^\mathsf{T}\mathbf{x}+b)$. For I receiving units it is $\mathbf{a} = f(\mathbf{W}^\mathsf{T}\mathbf{x}+\mathbf{b})$, with $\mathbf{W} \in \mathbb{R}^{D \times I}$. A constant extra input equal to one can absorb the bias into the weight matrix.

A shape check that catches transpose errors

For $D = 3$ features and $I = 5$ hidden units, $\mathbf{W}$ has shape 3×5. Therefore $\mathbf{W}^\mathsf{T}\mathbf{x}$ has shape $(5 \times 3)(3 \times 1) = 5 \times 1$, exactly matching $\mathbf{b}$. In a batch of four samples, $(5 \times 3)(3 \times 4) = 5 \times 4$: each column is still one sample.

Other useful operations are the componentwise product $(\mathbf{u} \odot \mathbf{v})_i = u_i v_i$, the inner product $\mathbf{u}^\mathsf{T}\mathbf{v}$, and the outer product $(\mathbf{u}\mathbf{v}^\mathsf{T})_{ij} = u_i v_j$. The squared Euclidean norm is $\|\mathbf{w}\|_2^2 = \sum_i w_i^2$; it is not the norm $\|\mathbf{w}\|_2$ itself.

0.7 Functions

The notation $g(\mathbf{x}; \boldsymbol{\theta})$ separates the input from the parameters that determine the model. Likewise $p(\mathbf{x}; \boldsymbol{\theta})$ is a probability mass function or density. A hidden activation such as tanh or ReLU acts elementwise on vectors. Softmax is different: each component depends on all logits through the shared denominator,

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

Subtracting $\max_j z_j$ from every logit leaves this expression unchanged and avoids unnecessarily large exponentials.

0.8 Generalized Functions

The names sound more complicated than the idea. A *functional* takes a function and gives back a number: risk gives a prediction rule a score. An *operator* turns one function into another, as differentiation does. Once a model is written using parameters $\boldsymbol{\theta}$, its risk is simply a function of those parameters, and we can differentiate it as usual.

0.9 Loss, Risk, and Regularization

A loss $\ell(\mathbf{y}, \widehat{\mathbf{y}})$ measures one prediction error. Population risk averages the loss under the data distribution; empirical risk averages observed errors:

$$R(\boldsymbol{\theta}) = \mathbb{E}[\ell(\mathbf{y}, g(\mathbf{x}; \boldsymbol{\theta}))], \qquad R_{\text{emp}}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{n=1}^{N} \ell(\mathbf{y}^{(n)}, g(\mathbf{x}^{(n)}; \boldsymbol{\theta})).$$

A regularized objective adds a penalty, for example $R_{\text{emp}} + \lambda \left\|\boldsymbol{\theta}\right\|_2^2 / 2$. This is also called structural risk. For sequences, a sample loss may itself sum several time-step losses; the chosen normalization must also be used in the gradient.

0.10 Derivatives and Matrix Layout

For $\mathbf{u} = f(\mathbf{x}) \in \mathbb{R}^I$ and $\mathbf{x} \in \mathbb{R}^D$, the Jacobian uses numerator layout:

$$\left[\frac{\partial \mathbf{u}}{\partial \mathbf{x}}\right]_{ij} = \frac{\partial u_i}{\partial x_j}, \qquad \frac{\partial \mathbf{u}}{\partial \mathbf{x}} \in \mathbb{R}^{I \times D}.$$

The derivative of a scalar loss with respect to a vector is consequently a row; its gradient is a column, $\nabla_{\mathbf{x}} \mathcal{L} = (\partial \mathcal{L} / \partial \mathbf{x})^{\mathsf{T}}$. For elementwise f, the Jacobian is diagonal: $\operatorname{diag}(f'(\mathbf{s}))$.

Matrix gradients: use the shape needed for the update

Here $[\nabla_{\mathbf{W}} \mathcal{L}]_{di} = \partial \mathcal{L} / \partial W_{di}$, so the gradient has the *same shape* as $\mathbf{W}$. Some matrix-calculus conventions write this derivative transposed. Our update is directly $\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} \mathcal{L}$; do not mix the two layouts in one calculation.

0.11 Distributions

The uniform distribution $\mathcal{U}(a, b)$ has mean $(a + b)/2$ and variance $(b - a)^2/12$. The normal distribution $\mathcal{N}(\mu, \sigma^2)$ has mean μ and variance σ^2. The binomial distribution $\operatorname{Bin}(n, p)$ counts successes in n independent trials; its mean is np and variance $np(1 - p)$. For $n = 1$ it becomes $\operatorname{Ber}(p)$, the binary random variable used to construct dropout masks.

1

Simple Recurrent Networks

The same word can mean different things depending on what came before it. The same goes for a sensor reading or a position in a biological sequence. A feedforward predictor $\widehat{\mathbf{y}} = g(\mathbf{x}; \boldsymbol{\theta})$ sees only its supplied input. A recurrent neural network (RNN) also receives a state carried from the preceding step. The index t may refer to physical time or simply to position in a sequence.

Rather than concatenate an entire sequence into a fixed-size vector, an RNN repeatedly applies one transition. It can therefore process different sequence lengths with the same parameter set. A fixed input window is another valid design, but it makes the directly visible history finite. A recurrent state has no preset time horizon, although its finite dimension and learning dynamics limit which information it actually retains.

A causal recurrent sequence model

A state transition $F_{\boldsymbol{\theta}}$ and readout $G_{\boldsymbol{\theta}}$ define

$$\mathbf{a}(t) = F_{\boldsymbol{\theta}}(\mathbf{x}(t), \mathbf{a}(t-1)), \qquad \widehat{\mathbf{y}}(t) = G_{\boldsymbol{\theta}}(\mathbf{a}(t)).$$

Given an initial state $\mathbf{a}(0)$, the prediction at t can depend on $\mathbf{x}(1), \ldots, \mathbf{x}(t)$, but not on later inputs. The same parameters are reused at every step.

The state is a learned summary of the past, not a stored copy of the full input history. Expressive-power results show what idealized recurrent systems can represent; they do not provide an algorithm that reliably learns such representations.

1.1 Jordan Network

A Jordan network feeds back its previous *prediction*. For input dimension D, hidden dimension I, and output dimension K, the forward pass is

$$\mathbf{s}(t) = \mathbf{W}^{\mathsf{T}}\mathbf{x}(t) + \mathbf{R}^{\mathsf{T}}\widehat{\mathbf{y}}(t-1) + \mathbf{b}, \tag{1.1}$$

$$\mathbf{a}(t) = f(\mathbf{s}(t)), \qquad \widehat{\mathbf{y}}(t) = \phi(\mathbf{V}^{\mathsf{T}}\mathbf{a}(t) + \mathbf{c}), \tag{1.2}$$

where $\mathbf{W} \in \mathbb{R}^{D \times I}$, $\mathbf{R} \in \mathbb{R}^{K \times I}$, and $\mathbf{V} \in \mathbb{R}^{I \times K}$. Biases have shapes I and K. The recurrent matrix maps *outputs* into the next hidden pre-activation; it is not a hidden-to-hidden matrix here. A starting output $\widehat{\mathbf{y}}(0)$ must also be supplied.

Why share weights?. Each time step uses the same $\mathbf{W}, \mathbf{R}, \mathbf{V}$. A sequence of a thousand steps creates a longer computation, not a thousand separately trained networks. Excluding biases, the Jordan network has $DI + KI + IK$ parameters, independent of sequence length. Updating one weight changes its behavior at every position.

Choose the readout for the task. A real-valued regression target usually uses an identity output activation, binary classification a sigmoid, and mutually exclusive multiclass classification softmax. The hidden nonlinearity f and output activation ϕ serve different purposes. Sigmoid and tanh are elementwise; softmax couples output components.

1.1.1 Teacher Forcing

During training, the true previous label is available. *Teacher forcing* replaces $\widehat{\mathbf{y}}(t-1)$ by $\mathbf{y}(t-1)$ in the recurrent input. This gives the network a correct context even when its previous prediction was poor. At generation time that ground-truth context is generally unavailable, so the model must consume its own predictions.

Learning with a correct previous answer

Imagine predicting the next item in a sequence while a tutor supplies the correct preceding item after each attempt. Each local prediction is easier to learn. When the tutor leaves, however, one wrong answer can become the context for another wrong answer. This is the training–inference mismatch created by teacher forcing; it does not mean the current target is allowed as an input to its own prediction.

The original local training procedure updates from each time step locally, without tracing error through earlier steps. That is a choice of learning algorithm, not a requirement of the Jordan architecture. Chapter 2 separates the network's forward memory from the reach of its training signal.

1.2 Elman Network

The next change is to feed back the hidden representation instead of the output. In the *fixed-feedback Elman variant we use here*, the feedback is the identity:

$$\mathbf{s}(t) = \mathbf{W}^{\mathsf{T}}\mathbf{x}(t) + \mathbf{a}(t-1) + \mathbf{b}, \qquad \mathbf{a}(t) = f(\mathbf{s}(t)), \qquad \widehat{\mathbf{y}}(t) = \phi(\mathbf{V}^{\mathsf{T}}\mathbf{a}(t) + \mathbf{c}).$$

Each hidden unit receives its own previous activation with fixed weight one. The recurrent connection introduces no trainable parameters, so the count without biases is $I(D+K)$. The state can preserve information that is not immediately useful for the current output; it no longer has to pass through a K-dimensional readout at every step.

Watch out for the name Elman

Here "Elman" means fixed identity feedback, and "fully recurrent" means a learned dense recurrent matrix. Elsewhere, Elman or simple RNN often also refers to the learned-matrix version. Furthermore, identity feedback *alone* does not preserve gradients: the derivative of f still lies on the temporal path.

1.3 Fully Recurrent Network

Allowing hidden units to exchange information across time replaces the fixed identity by a trainable $\mathbf{R} \in \mathbb{R}^{I \times I}$. The resulting network is the standard dense recurrence used in the following derivations.

Figure planned: Three feedback routes

Show Jordan, fixed-feedback Elman, and fully recurrent networks side by side. Highlight the delayed arrow from the output, the identity self-feedback, and the dense hidden-state feedback. Label each route with its matrix shape.

Figure 1.1: A comparison of what each architecture carries into the next step.

Forward equations and parameter shapes

$$\mathbf{s}(t) = \mathbf{W}^\mathsf{T}\mathbf{x}(t) + \mathbf{R}^\mathsf{T}\mathbf{a}(t-1) + \mathbf{b}, \tag{1.3}$$

$$\mathbf{a}(t) = f(\mathbf{s}(t)), \qquad \mathbf{z}(t) = \mathbf{V}^\mathsf{T}\mathbf{a}(t) + \mathbf{c}, \qquad \widehat{\mathbf{y}}(t) = \phi(\mathbf{z}(t)). \tag{1.4}$$

Here $\mathbf{W} : D \times I$, $\mathbf{R} : I \times I$, $\mathbf{V} : I \times K$. There are $I(D+I+K)$ weights, or $I(D+I+K)+I+K$ parameters including biases.

Under this convention, R_{ij} connects *previous* hidden unit i to *current* unit j: $s_j(t) = \sum_d W_{dj}x_d(t) + \sum_i R_{ij}a_i(t-1) + b_j$. This index interpretation will make the outer-product gradients in the next chapter easy to check.

Starting and ending a sequence. A common initial state is $\mathbf{a}(0) = \mathbf{0}$. A learned initial state or an application-specific context is also possible. Independent sequences normally start with fresh states; carrying state across unrelated examples creates an unintended dependence. Outputs can be evaluated at every step or only at selected positions, such as the end of a sequence.

A memory trace in three steps

Take a scalar recurrence $a(t) = \tanh(x(t) + 0.5a(t-1))$, with $a(0) = 0$, identity readout, and inputs $(1, 0, 0)$. Then

$$a(1) \approx 0.762, \qquad a(2) \approx 0.363, \qquad a(3) \approx 0.180.$$

The second and third inputs are zero, yet their states are not: the first input still influences them through feedback. Its effect weakens with each transition. State values show what is remembered in the forward pass; derivatives will tell us whether training can learn to preserve that information.

In an unrolled picture, the state at one position becomes an input to the next copy of the transition. These are repeated uses of the same cell, not independent cells. Dense inference needs a state of size I and approximately $DI + I^2 + IK$ arithmetic work per step; training additionally needs information for its backward pass.

1.4 ARMA

Recurrent structure is not unique to neural networks. In a scalar autoregressive model, recent observations help explain the next value. A moving-average component instead uses past

innovations, the unpredictable residuals $\varepsilon(t)$. Combining both gives

$$x(t) = c + \sum_{p=1}^{P} \alpha_p x(t-p) + \sum_{q=1}^{Q} \beta_q \varepsilon(t-q) + \varepsilon(t).$$

This is an ARMA(P, Q) model. Removing the innovation-history sum gives an AR model; removing the observation-history sum gives an MA model. We use α, β to avoid confusing these scalar coefficients with the neural-network matrices.

When innovations have conditional mean zero, the one-step conditional-mean forecast omits the *new* $\varepsilon(t)$ but uses the past observations and estimated innovations. For example, $c = 0$, $\alpha_1 = 0.8$, $\beta_1 = 0.2$, $x(t-1) = 2$, and $\varepsilon(t-1) = -0.5$ give $\widehat{x}(t) = 1.5$. The actual observation is $1.5 + \varepsilon(t)$. "Moving average" here does not mean an arithmetic average of the recent observations. The model is linear, whereas nonlinear recurrent networks can learn more general transitions.

1.5 NARX Recurrent Neural Networks

A nonlinear autoregressive model with exogenous inputs (NARX) combines explicit output lags with a window of external inputs:

$$\widehat{\mathbf{y}}(t) = g(\widehat{\mathbf{y}}(t-1), \ldots, \widehat{\mathbf{y}}(t-T_y), \mathbf{x}(t), \ldots, \mathbf{x}(t-T_x); \boldsymbol{\theta}).$$

The term *exogenous* distinguishes the supplied input sequence from the model's own predicted outputs. If g is a neural network, feeding its outputs back makes the full system recurrent. Jordan is the case $T_y = 1$, $T_x = 0$ with the corresponding one-hidden-layer map.

Explicit lags give a recent output a short route to the current prediction, rather than forcing it through every intervening transition. The input width of g is $KT_y + D(T_x + 1)$, so longer windows increase the first layer's parameter count. These shortcuts can ease learning over the chosen lag range, but do not guarantee nonvanishing gradients or solve arbitrarily long dependencies. Recursive outputs can still carry indirect information from beyond the explicit window.

1.6 Time-Delay Neural Networks

A time-delay neural network (TDNN) connects adjacent layers at selected temporal offsets. A causal layer can be written as

$$a_j^{[l]}(t) = f\left(b_j^{[l]} + \sum_i \sum_{q=0}^{Q} W_{ijq}^{[l]} a_i^{[l-1]}(t-d_q)\right), \qquad d_q \geq 0.$$

Each delay has its own weights, shared across positions. With consecutive offsets this is a one-dimensional convolutional layer in the usual deep-learning convention. Sparse offsets provide a temporal receptive field with gaps; stacking layers enlarges that field. Boundary positions require an explicit padding or cropping convention.

Unlike a hidden-state recurrence, a feedforward TDNN has a finite receptive field determined by its layers and delays. One can also learn delays with a softmax over candidate offsets. A softmax is a soft selection; obtaining a single discrete delay additionally requires concentration or a selection rule. This does not remove the cost of considering candidate delays.

2

Learning Algorithms for RNNs

An RNN can use the past when making a prediction. But how does it learn which old information was worth keeping? That is the problem of *temporal credit assignment*: a mistake now needs to teach the network what it should have done earlier. We will follow that error backward, work through a small example, and then see why long gaps make learning difficult.

2.1 Empirical Risk Minimization via Gradient Descent

Let sample n contain an input sequence and its targets. Here we average over sequences and sum over time:

$$R_{\text{emp}}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{n=1}^{N} \sum_{t=1}^{T_n} m_t^{(n)} \ell\big(\mathbf{y}^{(n)}(t), \widehat{\mathbf{y}}^{(n)}(t)\big). \tag{2.1}$$

The mask $m_t^{(n)}$ is one for a supervised position and zero for an unsupervised or padded position. With equal lengths and all masks one, every step contributes. Sequence averaging gives longer sequences more total weight when losses are summed. Averaging over valid time steps instead is a different normalization, which must also appear in the gradients.

The parameters influence not just the current output but the entire state trajectory. Gradient descent updates them using

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} R_{\text{emp}}, \qquad \eta > 0.$$

For a differentiable objective and a sufficiently small step at a point with nonzero gradient, the negative gradient is a descent direction. An arbitrary fixed learning rate need not decrease the loss, and a zero gradient need not be a minimum. Stochastic mini-batch updates need not improve the full training loss on every step either.

2.2 Backpropagation Through Time

Unrolling an RNN turns a loop into a chain of computations indexed by time. For a finite sequence this is an ordinary directed acyclic computation graph, so the chain rule applies just as in a feedforward network. Backpropagation through time (BPTT) is backpropagation on that graph.

The forward pass computes states and losses in chronological order. The backward pass moves from the final step to the first, combining the local output error with error arriving from the future. We hold the parameters fixed during this forward/backward calculation and update them after accumulating their gradients. All appearances of a shared parameter contribute to *one sum*, not to separate independently updated weights.

Figure planned: Unroll the cell; trace the two error routes

Draw three consecutive hidden states with inputs below and losses above. Use navy arrows for the forward pass and red arrows for gradients. At the middle pre-activation, show the local output contribution and the contribution arriving from the next state. Repeat identical W, R, V labels.

Figure 2.1: BPTT adds present and future error signals at each hidden state.

2.2.1 BPTT for the Fully Recurrent Network

Use the forward equations (1.3)–(1.4). For one sequence, write $\mathcal{L} = \sum_{t=1}^{T} \ell_t$, absorbing any supervision mask into ℓ_t. Define two *column* vectors:

$$\boldsymbol{\psi}(t) = \nabla_{\mathbf{z}(t)} \ell_t \in \mathbb{R}^K, \qquad \boldsymbol{\delta}(t) = \nabla_{\mathbf{s}(t)} \mathcal{L} \in \mathbb{R}^I.$$

An output delta $\boldsymbol{\psi}(t)$ measures sensitivity to the current logits. A hidden delta $\boldsymbol{\delta}(t)$ measures sensitivity of the *total sequence loss*, including future errors, to the current pre-activation.

Step 1: differentiate the readout. Let $\mathbf{e}(t) = \nabla_{\widehat{\mathbf{y}}(t)} \ell_t$. In general,

$$\boldsymbol{\psi}(t) = \mathbf{J}_{\phi}(\mathbf{z}(t))^{\mathsf{T}} \mathbf{e}(t).$$

For an elementwise output activation this becomes $\mathbf{e}(t) \odot \phi'(\mathbf{z}(t))$. For identity readout with half-squared error, $\boldsymbol{\psi}(t) = \widehat{\mathbf{y}}(t) - \mathbf{y}(t)$. Softmax with cross-entropy and a target distribution whose entries sum to one gives the same compact difference, although softmax itself is *not* elementwise. With a supervision mask, multiply this local delta by the mask.

Step 2: combine the two routes into a hidden unit. The activation $\mathbf{a}(t)$ affects the current logits through $\mathbf{V}^{\mathsf{T}}$ and the next pre-activation through $\mathbf{R}^{\mathsf{T}}$. The chain rule therefore gives

$$\nabla_{\mathbf{a}(t)} \mathcal{L} = \mathbf{V}\boldsymbol{\psi}(t) + \mathbf{R}\boldsymbol{\delta}(t+1).$$

Passing through the hidden nonlinearity multiplies each component by $f'(s_i(t))$.

The backward recurrence

$$\boldsymbol{\delta}(t) = f'(\mathbf{s}(t)) \odot \left[\mathbf{V}\boldsymbol{\psi}(t) + \mathbf{R}\boldsymbol{\delta}(t+1)\right], \qquad \boldsymbol{\delta}(T+1) = \mathbf{0}. \tag{2.2}$$

Evaluate this from $t = T$ down to 1. At an unsupervised step the local output term is zero, but future errors can still produce a nonzero hidden delta.

A hidden unit receives two kinds of feedback

The local term asks, "Did this state produce a good answer now?" The future term asks, "Did this state prepare useful information for later?" Ignoring the second question can train immediate predictions while failing to learn the memory needed for a delayed answer. Recurrence in the forward pass does not by itself provide temporal credit assignment.

Step 3: accumulate each shared weight's contributions. A weight affects its receiving pre-activation in proportion to the activation on the sending side. Since R_{ij} connects $a_i(t-1)$ to $s_j(t)$,

$$\frac{\partial \mathcal{L}}{\partial R_{ij}} = \sum_{t=1}^{T} a_i(t-1)\delta_j(t).$$

Writing all such component products together gives outer products. Our gradients have the same orientation as the stored matrices:

$$\nabla_{\mathbf{W}}\mathcal{L} = \sum_{t=1}^{T} \mathbf{x}(t)\boldsymbol{\delta}(t)^{\mathsf{T}} \qquad \in \mathbb{R}^{D\times I}, \tag{2.3}$$

$$\nabla_{\mathbf{R}}\mathcal{L} = \sum_{t=1}^{T} \mathbf{a}(t-1)\boldsymbol{\delta}(t)^{\mathsf{T}} \qquad \in \mathbb{R}^{I\times I}, \tag{2.4}$$

$$\nabla_{\mathbf{V}}\mathcal{L} = \sum_{t=1}^{T} \mathbf{a}(t)\boldsymbol{\psi}(t)^{\mathsf{T}} \qquad \in \mathbb{R}^{I\times K}. \tag{2.5}$$

Bias gradients are $\sum_t \boldsymbol{\delta}(t)$ and $\sum_t \boldsymbol{\psi}(t)$. If the initial state is trainable, its gradient is $\mathbf{R}\boldsymbol{\delta}(1)$; otherwise no update is made to it. If you see these outer products transposed elsewhere, check the matrix storage and derivative conventions. Our formulas have the same shapes as the weights they update.

A complete two-step gradient calculation

Use a scalar linear RNN $a_t = wx_t + ra_{t-1}$, $\widehat{y}_t = va_t$, with $a_0 = 0$, $(x_1, x_2) = (1, 0)$, $(w, r, v) = (1, 0.5, 1)$. Only the final step has a target, $y_2 = 1$, with $\mathcal{L} = \frac{1}{2}(\widehat{y}_2 - 1)^2$.

Forward: $a_1 = 1$, $a_2 = 0.5$, $\widehat{y}_2 = 0.5$, so $\mathcal{L} = 0.125$. Backward: $\psi_2 = -0.5$, $\delta_2 = v\psi_2 = -0.5$, and $\delta_1 = r\delta_2 = -0.25$. Thus

$$\frac{\partial \mathcal{L}}{\partial w} = x_1\delta_1 + x_2\delta_2 = -0.25, \qquad \frac{\partial \mathcal{L}}{\partial r} = a_0\delta_1 + a_1\delta_2 = -0.5, \qquad \frac{\partial \mathcal{L}}{\partial v} = a_2\psi_2 = -0.25.$$

Directly, $\widehat{y}_2 = vrw$ gives the same derivatives. The input at step 1 receives credit despite having no local target. If we cut the gradient between steps 1 and 2, the computed w gradient becomes zero in this example.

Mini-batches turn sums into matrix products. Stack samples in columns: $\mathbf{X}(t) : D \times B$, $\mathbf{A}(t) : I \times B$, $\boldsymbol{\Delta}(t) : I \times B$, and $\boldsymbol{\Psi}(t) : K \times B$. Deltas in each column initially refer to that sample's unaveraged loss. The empirical-risk gradients are

$$\nabla_{\mathbf{W}} R_{\text{batch}} = \frac{1}{B}\sum_t \mathbf{X}(t)\boldsymbol{\Delta}(t)^{\mathsf{T}}, \qquad \nabla_{\mathbf{R}} R_{\text{batch}} = \frac{1}{B}\sum_t \mathbf{A}(t-1)\boldsymbol{\Delta}(t)^{\mathsf{T}},$$

$$\nabla_{\mathbf{V}} R_{\text{batch}} = \frac{1}{B}\sum_t \mathbf{A}(t)\boldsymbol{\Psi}(t)^{\mathsf{T}}.$$

The columns share parameters, not state: one sample does not become another sample's temporal context. Full-batch learning uses every sample; stochastic single-sample learning uses one; mini-batches trade gradient variability for efficient parallel computation. If the loss has already been averaged before differentiation, do not divide its gradients by B again.

Cost and a practical recipe. With P_θ parameters, a dense recurrent forward/backward pass costs approximately $\mathcal{O}(TP_\theta)$ per sequence. Standard BPTT saves activations across time, giving storage that grows linearly with T, roughly $\mathcal{O}(T(I + K))$ beyond inputs and parameters. Checkpointing can trade recomputation for storage. The procedure is: initialize state; compute and save the forward trajectory; initialize the terminal delta; sweep backward and sum the outer products; apply the optimizer once. It is not necessary to construct full Jacobian matrices to run this reverse-mode calculation.

2.3 Regularization

A network that fits a long sequence is not necessarily one that generalizes. Regularization addresses overfitting; it is conceptually separate from whether gradients can travel through time.

2.3.1 Early Stopping

Fit parameters on a training split, monitor a validation criterion, and retain the checkpoint with the best validation score. Stop when it fails to improve for a chosen patience interval, rather than reacting to a single noisy fluctuation. Use the test set only for the final evaluation. For temporally related data, the split must respect the intended prediction setting; overlapping windows from the same sequence can otherwise leak information across splits.

2.3.2 Weight Decay

An L^2 penalty discourages large weights by minimizing

$$R_{\text{reg}}(\boldsymbol{\theta}) = R_{\text{emp}}(\boldsymbol{\theta}) + \frac{\lambda}{2} \|\boldsymbol{\theta}\|_2^2 .$$

For ordinary gradient descent on the penalized parameters,

$$\boldsymbol{\theta} \leftarrow (1 - \eta\lambda)\boldsymbol{\theta} - \eta \nabla R_{\text{emp}}(\boldsymbol{\theta}).$$

This multiplicative shrinkage explains the term weight decay. Its probabilistic interpretation comes from an isotropic Gaussian prior:

$$\log p(\boldsymbol{\theta}) = -\frac{\|\boldsymbol{\theta}\|_2^2}{2\sigma^2} + C.$$

Maximizing log likelihood plus log prior is equivalent to minimizing negative log likelihood plus a positive quadratic penalty. With a *summed* negative log likelihood, $\lambda = 1/\sigma^2$ in our $\lambda/2$ convention; averaging the likelihood over N samples rescales this coefficient to $1/(N\sigma^2)$. A Laplace prior analogously yields an L^1 penalty. Penalizing a matrix does not by itself guarantee good temporal gradients, and L^2 regularization is not generally identical to decoupled weight decay under an adaptive optimizer.

2.3.3 Dropout

Dropout randomly suppresses activations during training. Using the *keep probability* q and independent Bernoulli mask entries m_i, inverted dropout is

$$m_i \sim \text{Ber}(q), \qquad \widetilde{a}_i = \frac{m_i}{q} a_i, \qquad \mathbb{E}[\widetilde{a}_i \mid a_i] = a_i.$$

At inference, no mask or further scaling is applied. This matches the lecture's alternative convention of leaving surviving activations unscaled during training and scaling at inference, at the level of expected layer inputs. It does not imply that the nonlinear network's expected prediction is exactly preserved.

For recurrent networks, *where* a mask is applied and whether it is reused across time matters: randomly erasing the memory pathway can disrupt stored information. Recurrent dropout designs are treated with LSTM later. Always distinguish keep probability q from drop probability $1 - q$.

2.4 Other RNN Learning Algorithms

Full BPTT stores a trajectory and transports loss sensitivities backward. Alternatives either shorten that backward history or carry state sensitivities forward. Those choices affect exactness, storage, and computational cost in different ways.

2.4.1 Truncated Backpropagation Through Time

Truncated BPTT processes chunks of length τ. At a boundary, the state value may be passed to the next chunk while its computation history is *detached*. Gradients within that chunk treat the incoming state as a constant. After the chunk loss is differentiated, parameters can be updated and the next chunk processed. Other overlapping truncation schedules are possible; the important choice is which temporal derivative paths are cut.

Remembering is different from assigning credit

A carried state can still contain information from many chunks ago. Detachment does not erase that value. It erases the gradient route to the old operations that created it. The network can therefore use an old memory without receiving direct training credit for how it was written outside the backward window.

For ordinary chunked BPTT, activation storage is approximately $\mathcal{O}(\tau I)$ and work per chunk is $\mathcal{O}(\tau P_\theta)$. Processing T steps still costs about $\mathcal{O}(TP_\theta)$ overall. The gradient is truncated, generally biased relative to the full-sequence gradient, and need not even point in the same direction. Increasing τ extends the available credit-assignment range but costs memory.

2.4.2 Real-Time Recurrent Learning

Real-time recurrent learning (RTRL) reverses the bookkeeping strategy. It asks at each step how the current state changes with *every parameter*, before a loss is known. For the generic transition $\mathbf{a}(t) = F(\mathbf{a}(t-1), \mathbf{x}(t); \boldsymbol{\theta})$, define

$$\mathbf{P}(t) = \frac{\partial \mathbf{a}(t)}{\partial \boldsymbol{\theta}} \in \mathbb{R}^{I \times P_\theta}, \quad \mathbf{J}_a(t) = \frac{\partial F}{\partial \mathbf{a}(t-1)}, \quad \mathbf{B}_\theta(t) = \left.\frac{\partial F}{\partial \boldsymbol{\theta}}\right|_{\mathbf{a}(t-1) \text{ fixed}}.$$

The total derivative is the forward recursion

$$\mathbf{P}(t) = \mathbf{J}_a(t)\mathbf{P}(t-1) + \mathbf{B}_\theta(t). \tag{2.6}$$

With a fixed parameter-independent initial state, $\mathbf{P}(0) = 0$. The current loss gradient is then $\mathbf{P}(t)^\top \nabla_{\mathbf{a}(t)} \ell_t$, plus any explicit dependence of the readout/loss on parameters. This is exact differentiation of the recurrent computation for fixed shared parameters. Online updates change the trajectory as parameters change; they should not be confused with recomputing a fixed-parameter full-sequence batch gradient.

For our pre-activation form, the recurrent-weight sensitivity is particularly concrete:

$$\frac{\partial s_j(t)}{\partial R_{uv}} = a_u(t-1)\mathbb{1}_{\{j=v\}} + \sum_i R_{ij} f'(s_i(t-1)) \frac{\partial s_i(t-1)}{\partial R_{uv}}.$$

The first term is the direct use of R_{uv} now; the sum accounts for its influence on past states. RTRL carries these sensitivities forward instead of saving all old activations. With $P_\theta = \mathcal{O}(I^2)$, storing $\mathbf{P}$ costs $\mathcal{O}(I^3)$ and updating it densely costs $\mathcal{O}(I^4)$ *per step*. Storage is independent of T; total computation is not.

2.4.3 Schmidhuber's Approach

This hybrid uses blocks of roughly I steps. It uses BPTT inside a block and carries sensitivity information between blocks in an RTRL-like manner. Unlike ordinary truncation, its purpose is to preserve gradient information across the boundary rather than discard it.

For a dense network, a boundary sensitivity update costs $\mathcal{O}(I^4)$ but occurs only once per I steps, yielding an amortized $\mathcal{O}(I^3)$ per step and $\mathcal{O}(TI^3)$ total. Ordinary within-block BPTT is lower order at $\mathcal{O}(I^3)$ per block. Only a block's activations must be retained, together with the boundary sensitivities; the method should not be described as having only block-activation storage. The point is to keep credit flowing across blocks. When comparing the costs, check whether a number is per step, per block, or for the whole sequence.

2.5 The Vanishing Gradient Problem

Even exact BPTT can return a learning signal too small to use, or so large that optimization becomes unstable. This is a property of the derivatives along the trajectory, not an error in applying the chain rule.

2.5.1 Long-Term Dependencies

Suppose an early input at t_0 determines the correct answer at a much later t. The network must learn to write that information, preserve it, and read it out when needed. A final loss teaches the early transition only through the state-to-state derivative linking those times.

For the pre-activation state in Equation (1.3), one step has Jacobian

$$\mathbf{J}_s(t) = \frac{\partial \mathbf{s}(t)}{\partial \mathbf{s}(t-1)} = \mathbf{R}^\mathsf{T} \operatorname{diag}(f'(\mathbf{s}(t-1))).$$

Across l steps the chain rule gives the ordered product

$$\frac{\partial \mathbf{s}(t)}{\partial \mathbf{s}(t-l)} = \mathbf{J}_s(t)\mathbf{J}_s(t-1)\cdots\mathbf{J}_s(t-l+1). \tag{2.7}$$

There are exactly l factors, and their order matters. Writing all of them for $l = 2$ is a useful check against off-by-one errors. A distant error signal must traverse this whole product before it reaches an early computation.

2.5.2 Vanishing and Exploding Gradients

For one hidden unit, the temporal derivative reduces to a product of scalar gains:

$$\frac{\partial s(t)}{\partial s(t-l)} = \prod_{k=0}^{l-1} \left[r f'(s(t-k-1)) \right].$$

If the gains repeatedly have magnitude below one, distant effects shrink; repeated amplification can make them grow. Signs may flip the gradient direction, so stability is about *absolute magnitudes*, not whether the signed number is less than one.

Small per-step changes become large over time

For a constant scalar gain g, the sensitivity after 50 steps is g^{50}. With $g = 0.9$ it is about 0.00515; with $g = 1.1$ it is about 117.4; with $g = 1$ it stays one. For a sigmoid unit, $f'(s) \leq 1/4$, so a recurrent weight of one gives a gain no larger than $1/4$ per step. A fixed recurrent connection therefore does not automatically create a stable memory pathway.

Figure planned: Gradient strength versus temporal distance

Plot absolute sensitivity on a logarithmic vertical axis for constant gains 0.9, 1.0, and 1.1 over 0–50 steps. Label the three endpoints and use direct curve labels. Include a small chain of transitions to connect the plot to the Jacobian product.

Figure 2.2: Repeated attenuation or amplification separates temporal gradient scales exponentially.

For several hidden units, the operator norm supplies a useful sufficient bound:

$$\left\| \frac{\partial \mathbf{s}(t)}{\partial \mathbf{s}(t-l)} \right\|_2 \leq \prod_{k=0}^{l-1} \left\| \mathbf{J}_s(t-k) \right\|_2 .$$

If every factor is bounded by a common $\rho < 1$, the product vanishes at least as fast as ρ^l. Conversely, a bound greater than one only *allows* growth; it does not prove every direction explodes. For time-varying nonlinear systems, looking only at individual eigenvalues is insufficient. Saturated activations can attenuate the signal even when the recurrent matrix is well scaled.

A stable linear path. With identity activation, the temporal Jacobian is simply $\mathbf{R}^\mathsf{T}$. An orthogonal recurrent matrix satisfies $\mathbf{R}^\mathsf{T}\mathbf{R} = \mathbf{I}$ and preserves Euclidean gradient norms along that linear recurrence. This establishes norm preservation for the temporal path, not a guarantee that the total parameter gradient or the whole optimization problem is well behaved.

The special choice $\mathbf{R} = \mathbf{I}$ also leaves stored coordinates unchanged in the absence of new input. Another orthogonal matrix can rotate the representation while preserving its norm, making the readout depend on elapsed time. Identity recurrence thus provides a particularly simple route to preserving information across different delays. Once a nonlinear activation is inserted into that path, its derivatives must again be considered.

What gradient clipping fixes. As a practical addition, global norm clipping replaces a gradient $\mathbf{g}$ by $\mathbf{g} \min(1, c/\left\|\mathbf{g}\right\|_2)$ for a threshold $c > 0$ (leaving a zero gradient unchanged). It limits large updates; it cannot reconstruct an already vanished signal. The architectural challenge is to provide a stable memory path while retaining useful nonlinear computation, which motivates LSTM.

2.5.3 Focused Backpropagation

The focused architecture separates nonlinear input processing from linear temporal integration:

$$\mathbf{s}(t) = \mathbf{W}^\mathsf{T}\mathbf{x}(t), \qquad \mathbf{c}(t) = f(\mathbf{s}(t)) + \mathbf{D}\mathbf{c}(t-1), \qquad \mathbf{D} = \operatorname{diag}(d_1, \ldots, d_I).$$

Here $\mathbf{c}(t)$ denotes the context state, not the readout bias $\mathbf{c}$ in the earlier architecture. Each coordinate accumulates new evidence and retains a scaled copy of its previous content. For a

loss only at the final step and $\boldsymbol{\gamma}(t) = \nabla_{\mathbf{c}(t)}\mathcal{L}$,

$$\boldsymbol{\gamma}(t) = \mathbf{D}^{\mathsf{T}}\boldsymbol{\gamma}(t+1) = \left(\mathbf{D}^{\mathsf{T}}\right)^{T-t}\boldsymbol{\gamma}(T).$$

The temporal path no longer repeatedly multiplies by f'. Nonlinearity is used to write information, while recurrence carries it linearly. A decay $0 < d_i < 1$ still attenuates memory and gradients; $d_i = 1$ preserves that coordinate along the direct path. For arbitrary real diagonal entries, norm preservation requires $|d_i| = 1$; choosing all $d_i = 1$ additionally avoids sign alternation and yields unchanged storage. LSTM will add learned control over writing, retaining, and reading such memory.

Before moving on. Be able to derive the two contributions to a hidden delta, check the shapes of all three weight gradients, and explain why truncation cuts credit without necessarily clearing the state. Finally, distinguish the existence of an exact gradient from its numerical usefulness: the Jacobian product is the link between learning algorithms and memory architecture.

3

Long Short-Term Memory

Suppose a sentence starts with "The keys to the old house ..." and only much later reaches its verb. To choose "are" rather than "is", the model needs to remember the plural subject while ignoring plenty of distracting words. An ordinary RNN can carry that information forward, but learning *what to keep* gets difficult when its gradient keeps shrinking along the way.

LSTM gives memory a more direct route through time. The main idea is simple: keep a running memory, then learn when to write into it and when to use it. Nonlinear processing still happens, but it does not have to sit on every step of the direct memory path.

A scratchpad with controls

Think of the cell state as a scratchpad. The input gate decides how much new information to write. The output gate decides how much of the scratchpad to show to the rest of the network. Later, the forget gate lets us erase or fade old entries. These are useful roles to remember, not literal labels learned by individual cells: in a trained network, information can be spread across many coordinates.

First, build the memory. A scalar accumulator has the update $c_t = c_{t-1} + z_t$. Its direct derivative with respect to the previous state is one: a change in the old state survives the next addition. The problem is that it writes *everything*. Adding an input gate gives $c_t = c_{t-1} + i_t z_t$. Now the model can turn down irrelevant updates without repeatedly squashing the stored state itself.

We write $\mathbf{c}_t$ for the cell state and $\mathbf{h}_t$ for the exposed hidden output. Both have I components. Keeping these symbols separate is worth it: $\mathbf{h}_t$ is not a label, and it is not the same thing as the memory. The candidate $\mathbf{z}_t$ is the proposed new content. Below, g transforms the candidate and u transforms the cell before readout; normally both are tanh.

The basic LSTM: write and read, without a forget gate

For $\alpha \in \{i, o, z\}$, first compute

$$\mathbf{s}_{\alpha,t} = \mathbf{W}_\alpha^\top \mathbf{x}_t + \mathbf{R}_\alpha^\top \mathbf{h}_{t-1} + \mathbf{b}_\alpha.$$

Then

$$\mathbf{i}_t = \sigma(\mathbf{s}_{i,t}), \qquad \mathbf{o}_t = \sigma(\mathbf{s}_{o,t}), \qquad \mathbf{z}_t = g(\mathbf{s}_{z,t}), \tag{3.1}$$

$$\mathbf{c}_t = \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{z}_t, \qquad \mathbf{h}_t = \mathbf{o}_t \odot u(\mathbf{c}_t). \tag{3.2}$$

Each $\mathbf{W}_\alpha$ is $D \times I$, each $\mathbf{R}_\alpha$ is $I \times I$, and each bias has I entries. A separate readout turns $\mathbf{h}_t$ into a prediction.

A sigmoid gate lies between zero and one. Near zero means "let almost nothing through"; near one means "let almost everything through". We often say open or closed, but ordinary gates are smooth values rather than binary switches. Their values are computed from the input and previous hidden output, and their weights are learned from the task loss. No one supplies target gate values.

The cell update *adds* new content; it does not run the old state through another tanh. The readout may use $\tanh(\mathbf{c}_t)$ without changing that stored state. This direct additive route is the *constant error carousel* (CEC). Without a forget gate, its direct derivative is the identity. That statement concerns this particular route: the complete network also has paths through gates and recurrent weights, so LSTM does not make every gradient automatically stable.

Figure planned: An LSTM cell you can read from left to right

Draw a thick horizontal cell-state path. Add candidate content through the input gate; branch down through tanh and the output gate to h. Put the optional forget gate on the state path in a dashed outline. Keep c and h visually distinct and label the operations keep, write, and read.

Figure 3.1: The cell state carries memory; the hidden output reveals a gated view of it.

Why use a signed candidate?. With $g = \tanh$, a candidate can be positive or negative, so the cell can accumulate evidence for and against something. A sigmoid candidate is always positive. Without forgetting, that can produce upward drift, although it can be useful for detecting whether a rare event has occurred. Also, $\mathbf{c}_t$ itself is not restricted to $[-1, 1]$: only its tanh readout is bounded.

3.1 Backpropagation for LSTM

There is no separate magic optimizer for LSTM. We can unroll it and use BPTT, just as before. The extra bookkeeping comes from having two recurrent outputs, $\mathbf{h}_t$ and $\mathbf{c}_t$, plus several branches inside the cell. At an addition, the gradient goes to both inputs; at a product, each input receives the upstream gradient multiplied by the other input.

Let $\mathbf{r}_t$ be the gradient arriving at $\mathbf{h}_t$ from the current prediction loss. For example, with readout logits $\mathbf{V}^\mathsf{T}\mathbf{h}_t + \mathbf{b}_{\text{out}}$, it is $\mathbf{r}_t = \mathbf{V}\boldsymbol{\psi}_t$. Define $\mathbf{d}_t^h = \nabla_{\mathbf{h}_t}\mathcal{L}$, $\mathbf{d}_t^c = \nabla_{\mathbf{c}_t}\mathcal{L}$, and pre-activation deltas $\boldsymbol{\delta}_{\alpha,t}$. All are columns. Here we derive the basic cell without peepholes or forgetting.

1. Gather the errors reaching the hidden output. The next step uses $\mathbf{h}_t$ in three affine maps, so it sends three contributions back:

$$\mathbf{d}_t^h = \mathbf{r}_t + \mathbf{R}_i\boldsymbol{\delta}_{i,t+1} + \mathbf{R}_o\boldsymbol{\delta}_{o,t+1} + \mathbf{R}_z\boldsymbol{\delta}_{z,t+1}.$$

This is the same "present plus future" idea as in a simple RNN, now with more branches.

2. Follow the readout and the cell-state path. The output gate receives the exposed cell value as its multiplicative factor. The cell state receives a contribution from the current hidden output and another directly from the future cell:

$$\boldsymbol{\delta}_{o,t} = \mathbf{d}_t^h \odot u(\mathbf{c}_t) \odot \mathbf{o}_t \odot (1 - \mathbf{o}_t), \tag{3.3}$$

$$\mathbf{d}_t^c = \mathbf{d}_t^h \odot \mathbf{o}_t \odot u'(\mathbf{c}_t) + \mathbf{d}_{t+1}^c. \tag{3.4}$$

The second term is the key: the direct error from the next cell is *added*, not repeatedly multiplied by a squashing derivative.

3. Differentiate the write operation. From $\mathbf{i}_t \odot \mathbf{z}_t$, we get

$$\boldsymbol{\delta}_{i,t} = \mathbf{d}_t^c \odot \mathbf{z}_t \odot \mathbf{i}_t \odot (1 - \mathbf{i}_t), \tag{3.5}$$

$$\boldsymbol{\delta}_{z,t} = \mathbf{d}_t^c \odot \mathbf{i}_t \odot g'(\mathbf{s}_{z,t}). \tag{3.6}$$

For $g = u = \tanh$, use $\tanh'(s) = 1 - \tanh^2(s)$. For each sigmoid gate use $\sigma'(s) = \sigma(s)(1 - \sigma(s))$. Initialize the future deltas and $\mathbf{d}_{T+1}^c$ to zero, and process $t = T, T-1, \ldots, 1$.

4. Turn deltas into weight gradients. Nothing new is needed here: it is still sender activation times receiving delta,

$$\nabla_{\mathbf{W}_\alpha} \mathcal{L} = \sum_t \mathbf{x}_t \boldsymbol{\delta}_{\alpha,t}^\top, \qquad \nabla_{\mathbf{R}_\alpha} \mathcal{L} = \sum_t \mathbf{h}_{t-1} \boldsymbol{\delta}_{\alpha,t}^\top, \qquad \nabla_{\mathbf{b}_\alpha} \mathcal{L} = \sum_t \boldsymbol{\delta}_{\alpha,t}.$$

These matrices have the same shapes as the parameters they update. Batch averaging and the readout gradients work exactly as in Chapter 2.

A later loss can teach an earlier write

Take two scalar steps with $c_0 = 0$, no forgetting, and an identity cell readout. At step 1, $i_1 = 0.5$ and $z_1 = 0.8$; at step 2, the write is zero. Thus $c_1 = c_2 = 0.4$. Let the final output gate be $o_2 = 0.5$, so $h_2 = 0.2$, and use $\mathcal{L} = \frac{1}{2}(h_2 - 1)^2$.

The output gradient is -0.8. Along the direct memory path, $d_2^c = (-0.8)(0.5) = -0.4$ and $d_1^c = -0.4$. The first input gate therefore receives pre-activation delta

$$\delta_{i,1} = (-0.4)(0.8)(0.5)(1 - 0.5) = -0.08.$$

Increasing that gate's pre-activation would write more useful content and lower the loss. This toy example holds the other gate paths fixed so we can see the memory route on its own.

3.1.1 The LSTM Learning Method

The original LSTM training method deliberately stopped some gradient paths through the gates and candidate computations, while preserving the long path through the cell state. In the equations above, this removes the future gate/candidate terms from $\mathbf{d}_t^h$, leaving the local $\mathbf{r}_t$. The gates still learn from their local deltas; their recurrent influence is just not traced further back along those cut paths.

This is an approximate gradient for the full graph. It is also different from chopping time into fixed-length BPTT chunks: the memory-state path can remain long while selected *types of connection* are cut. Ordinary full BPTT keeps all these paths. Keep the architecture and the chosen training rule separate in your head.

3.2 Forget Gate

Keeping information forever is not always helpful. After a sentence, episode, or event ends, old content may get in the way. A forget gate controls how much of the previous cell survives:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f^\top \mathbf{x}_t + \mathbf{R}_f^\top \mathbf{h}_{t-1} + \mathbf{b}_f).$$

Despite its name, its value is a *keep fraction*: $f = 1$ means keep, $f = 0$ means erase. This avoids the confusing phrase "closing the forget gate".

Vanilla LSTM: keep, write, read

Compute the input gate, output gate, and candidate as before, plus the forget gate. Then

$$\mathbf{c}_t = \underbrace{\mathbf{f}_t \odot \mathbf{c}_{t-1}}_{\text{keep old content}} + \underbrace{\mathbf{i}_t \odot \mathbf{z}_t}_{\text{write new content}}, \tag{3.7}$$

$$\mathbf{h}_t = \underbrace{\mathbf{o}_t \odot \tanh(\mathbf{c}_t)}_{\text{show part of the memory}}. \tag{3.8}$$

A layer with I cells and D inputs has $4I(D + I + 1)$ parameters with one bias per gate/candidate unit, excluding the task readout. The basic three-branch cell has $3I(D + I + 1)$.

One complete cell update

Suppose $c_{t-1} = 2$, $f_t = 0.9$, $i_t = 0.25$, $z_t = -0.8$, and $o_t = 0.5$. We retain $0.9 \cdot 2 = 1.8$, add $0.25(-0.8) = -0.2$, and obtain $c_t = 1.6$. The exposed output is $h_t = 0.5 \tanh(1.6) \approx 0.461$.

The old memory was only partly forgotten, the candidate reduced it, and the output gate revealed only part of its transformed value. These are three different operations.

How long does a memory last?. Along the direct state path, with the gates held fixed, a coordinate's sensitivity over l steps is $\prod_{k=0}^{l-1} f_{t-k}$. If $f = 0.9$ throughout, only about 0.005 remains after 50 steps; if $f = 0.99$, about 0.605 remains. A gate that looks "almost open" at one step can still cause strong decay over a long sequence. For constant $0 < f < 1$, its half-life is $\log(1/2)/\log f$ steps.

A positive forget bias starts the network closer to keeping information: $\sigma(0) = 0.5$, $\sigma(1) \approx 0.731$, and $\sigma(3) \approx 0.953$. This is an initialization preference, not a guarantee of long memory. Input-dependent gates will change it.

What changes in backprop?. Add $\mathbf{R}_f \boldsymbol{\delta}_{f,t+1}$ to $\mathbf{d}_t^h$ and replace the cell-state recurrence by

$$\mathbf{d}_t^c = \mathbf{d}_t^h \odot \mathbf{o}_t \odot (1 - \tanh^2(\mathbf{c}_t)) + \mathbf{f}_{t+1} \odot \mathbf{d}_{t+1}^c.$$

The new delta is

$$\boldsymbol{\delta}_{f,t} = \mathbf{d}_t^c \odot \mathbf{c}_{t-1} \odot \mathbf{f}_t \odot (1 - \mathbf{f}_t).$$

Its weight gradients use the same outer products as the other branches. These equations assume no peephole connections; peepholes add further paths to the cell-state gradient.

Coupling input and forgetting. Setting $\mathbf{f}_t = 1 - \mathbf{i}_t$ gives

$$\mathbf{c}_t = (1 - \mathbf{i}_t) \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{z}_t.$$

Each coordinate interpolates between old and proposed content. This saves a gate, but removes the freedom to retain everything while also adding a large update. A related trade-off appears again in the GRU.

3.3 Tricks of the Trade

These ideas are best understood as changes to *how the model uses its memory*. They are not a checklist to apply blindly. Start with a clear task, then choose the modification that addresses an actual difficulty.

Focused and lightweight LSTM. A focused cell gives the input and the previous hidden output different jobs:

$$\mathbf{i}_t = \sigma(\mathbf{R}_i^\top \mathbf{h}_{t-1} + \mathbf{b}_i), \quad \mathbf{o}_t = \sigma(\mathbf{R}_o^\top \mathbf{h}_{t-1} + \mathbf{b}_o), \quad \mathbf{z}_t = g(\mathbf{W}_z^\top \mathbf{x}_t + \mathbf{b}_z).$$

It uses the basic no-forget memory update. The candidate detects input patterns; the gates decide, from the current context, whether to store or expose them. This uses $2I^2 + DI$ weights before biases, rather than $3I^2 + 3DI$ for the three-branch cell. The lightweight variant also removes the output gate. With sigmoid candidates and no forgetting, each memory coordinate can only increase: useful for accumulating positive detections, limiting if evidence needs to cancel.

Gate biases and drift. A negative input-gate bias makes the cell reluctant to write at first. It can help when relevant events are rare and irrelevant updates would otherwise accumulate. Extremely negative biases also saturate the sigmoid, making that reluctance slow to unlearn. Different biases across cells can encourage different initial write/read behavior; they do not guarantee an orderly ranking of learned concepts.

Choose what a cell should be able to represent. A tanh candidate allows signed evidence. A sigmoid candidate suits nonnegative event accumulation. A linear candidate avoids a bounded input activation but allows large updates. A linear cell readout can make counting easier because large counts are not squeezed towards ± 1. Scaling the candidate amplifies writes; scaling the readout amplifies what other units see. These changes solve different problems.

Counting versus detecting

"Did a stop signal appear?" and "How many stop signals appeared?" need different memories. A detector may only need to cross a threshold once. A counter needs distinct values for one, two, ten, and twenty events. A saturated readout can make those large counts look almost identical, even when the internal cell values differ.

Ticker steps: a little time to process. For an end-of-sequence prediction, append a few designated empty or repeated inputs before asking for the answer. The network gets extra recurrent computation steps to combine what it has stored. These *ticker steps* add computation, not new observations. Training and inference need a consistent convention for them.

Time information and input windows. If season or phase matters, supply it explicitly. For a period P, $\sin(2\pi t/P)$ and $\cos(2\pi t/P)$ encode phase without a discontinuity between the last and first phase. Localized basis functions or binary time features are other options. A short input window can make a local pattern, such as a sequence motif, visible all at once; the recurrent memory can then store the detected pattern rather than discovering it one symbol at a time. Windowing increases input size and must not introduce unavailable future inputs in a causal task.

Separate memory from computation. One can couple a recurrent processing layer to LSTM cells: the processor interprets the input, writes results into memory, and reads memory back later. Richer designs also connect gate outputs to other gates. They offer more ways to compute, but add gradient paths and make training harder to reason about. The useful idea is to let one part perform transformations while another preserves information across time.

Match the loss to the information available. A sequence label may only become predictable near the end. Demanding the final answer at every earlier step can create conflicting training signals. Conversely, a forecasting task really does need intermediate predictions. Decide where supervision makes sense, rather than assuming that more loss terms are always better. Separate models or task heads can reduce interference when different predictions need incompatible memories, at the cost of more parameters.

The ordinary training choices still count. Input scaling, enough cells to discover useful patterns, and a sensible learning-rate schedule can matter as much as a clever gate variant. Fit scaling statistics on training data. Transforming outliers can help, but should preserve information the task needs. For regression, target scaling can avoid a poorly chosen saturated readout; it is not a reason to replace classification labels by arbitrary values. Single-sequence updates, mini-batches, and changing the learning rate are options to compare on validation data, not universal winners.

3.4 Dropout for LSTM

Randomly wiping the cell state is a bad way to preserve a memory. If each step independently keeps a particular state coordinate with probability q, the chance of an uninterrupted route over T steps is q^T. Even $q = 0.9$ gives only about 0.005 over 50 steps. This calculation refers to directly zeroing that state route, not to every technique called recurrent dropout.

Drop inputs, keep the memory route. One simple option is dropout on non-recurrent inputs. With keep probability q and mask $\mathbf{m}_t \in \{0,1\}^D$, use $\widetilde{\mathbf{x}}_t = \mathbf{m}_t \odot \mathbf{x}_t/q$ during training. The mask has dimension D, because it acts on inputs, not on the I cells. Inverted scaling means no extra scaling is needed at inference. This regularizes incoming information without directly clearing the cell state.

Zoneout: sometimes keep the old state. Let $\mathbf{c}_t^{\text{new}}, \mathbf{h}_t^{\text{new}}$ be the normally computed next states. An update mask gives

$$\mathbf{c}_t = (1 - \mathbf{m}_t^c) \odot \mathbf{c}_{t-1} + \mathbf{m}_t^c \odot \mathbf{c}_t^{\text{new}}, \qquad \mathbf{h}_t = (1 - \mathbf{m}_t^h) \odot \mathbf{h}_{t-1} + \mathbf{m}_t^h \odot \mathbf{h}_t^{\text{new}}.$$

A zero mask entry means “skip this update”, not “replace the memory with zero”. For a no-forget cell, the first equation reduces to masking the new write. A retained coordinate also supplies an identity gradient route. The exact masking and inference conventions are part of the chosen variant; unlike inverted dropout, these identity-preserving masks are not divided by the keep probability.

3.5 Application Examples

The same memory mechanism can support very different tasks. For each example, ask three questions: what arrives at each step, what has to remain in memory, and when does the model produce an answer?

3.5.1 Sequence-to-Sequence Learning with LSTM

In an *aligned* task, each input position has a corresponding output: a word and its part-of-speech tag, for instance. A recurrent state supplies context, so the same word can receive different tags in different sentences. Equal input and output lengths alone do not imply alignment; the matching positions must also make sense for the task.

Translation is different. The output may have another length and word order. An encoder reads the source sequence; its final state initializes a decoder, possibly through a learned projection. The decoder then predicts one token at a time, starting with a start token and stopping at an end token. Both **c** and **h** may be transferred. The decoder loss trains the encoder too, because the state-transfer operation is differentiable.

$$p(y_{1:S} \mid x_{1:T}) = \prod_{j=1}^{S} p(y_j \mid y_{<j}, x_{1:T}).$$

During teacher-forced training, the decoder receives the true previous output token. During generation, it receives a token selected from its own prediction. There is no requirement that $S = T$.

Figure planned: From one final memory to a new sequence

Show an encoder reading three source tokens, transferring its c and h to a decoder, and a decoder producing four tokens including EOS. Mark teacher-forced previous targets versus generated previous tokens. Highlight the fixed-size bridge that all source information must cross.

Figure 3.2: An encoder-decoder model can change sequence length and order, but its fixed-size bridge can become a bottleneck.

Why embeddings help. A token index can look up a trainable vector rather than enter as a huge one-hot vector. With vocabulary size V and embedding width d, an embedding table has Vd entries. Word-level inputs shorten sequences but need a large vocabulary; character-level inputs need fewer symbols but more steps. CBOW predicts a word from its surrounding context; skip-gram predicts surrounding words from the center word. Embeddings capture patterns useful for their training objective, not guaranteed human-like meanings.

How do we score a translation?. BLEU combines clipped n-gram precisions with a brevity penalty:

$$\text{BLEU} = \text{BP} \exp\left(\sum_{n=1}^{N_g} w_n \log p_n\right), \qquad \sum_n w_n = 1.$$

Clipping prevents a candidate from receiving unlimited credit for repeating a reference word. With candidate length $c > 0$ and effective reference length r, BP $= 1$ for $c \geq r$, otherwise $\exp(1 - r/c)$. Corpus-level overlap is useful for comparison, but a valid paraphrase may receive little credit. BLEU is not a reliable judgment of the meaning of one sentence; zero precisions and smoothing conventions also matter.

3.5.2 Generating Sequences with LSTMs

A next-token model becomes a generator by feeding its output back as the next input. Given a prefix, compute a distribution, choose a token, append it, and repeat. Greedy selection always chooses the largest probability; sampling can produce variety. With temperature $\tau > 0$, $p_k = \text{softmax}(\boldsymbol{\ell}/\tau)_k$: a lower temperature sharpens the distribution, while a higher one flattens it.

Prediction and generation use the same model

After reading "ca", a character model might assign $p(t) = 0.6$, $p(r) = 0.3$, and $p(n) = 0.1$. Sampling "r" gives the new prefix "car", which changes the next distribution. During training on "cat", the correct next character is still "t"; the cross-entropy contribution is $-\log 0.6 \approx 0.511$. Generation chooses from a distribution; training teaches the distribution.

A byte-level model can learn text, markup, and recurring syntax using a small alphabet, but plausible syntax does not imply correct facts, working code, or valid mathematics. Continuous generation, such as handwriting coordinates, uses a distribution over real-valued outputs rather than a token softmax.

Next-token learning can also discover useful internal features. For example, a review model may track positive or negative sentiment because that helps predict upcoming text. A later classifier can use its state as an input feature. Such a feature emerging in an experiment does not mean every LSTM will dedicate one interpretable cell to sentiment. Likewise, carrying states over several training chunks can expose the model to long context even when the gradient is truncated within each chunk.

3.5.3 Image Captioning

An image encoder turns a picture into visual features; a recurrent decoder uses them while generating a caption. Memory tracks what has already been said and what is still relevant. A single global image vector has the same bottleneck as a final sentence state. Spatial attention improves the design by letting the decoder consult different image regions for different words, which we will develop in Chapter 4.

3.5.4 Learning to Learn Using LSTM

Here one sequence represents a *new learning task*. At step t, the network sees $(\mathbf{x}_t, \mathbf{y}_{t-1})$ and predicts $\mathbf{y}_t$. The previous label tells it how an earlier prediction should have turned out. Training across many such tasks teaches a useful state-update strategy.

Within a new task, the network can adapt its predictions by changing its *state* while its learned weights stay fixed. The weights contain the learning strategy; the state contains what it has learned about this particular task. Reset the task state between unrelated tasks. Supplying $\mathbf{y}_t$ before predicting it would make the exercise trivial and leak the answer.

3.5.5 Rainfall-Runoff Modelling

Rain now does not necessarily become river discharge now: soil moisture, snow storage, and earlier weather matter. An LSTM can process weather inputs and learn a state useful for predicting runoff. Static catchment features can tell it which kind of landscape it is dealing with. This is a good memory example, but a cell is not automatically a physically meaningful water reservoir.

A common evaluation score is Nash–Sutcliffe efficiency,

$$\mathrm{NSE} = 1 - \frac{\sum_t (q_t - \widehat{q}_t)^2}{\sum_t (q_t - \overline{q})^2}.$$

For nonconstant observations, one is perfect, zero matches the observed-mean baseline, and negative values are worse than that baseline. Test catchments and time periods determine what kind of generalization is being measured.

3.5.6 Talking Heads

A sequence model can map speech features or phoneme labels to facial movement. The same sound may be produced differently depending on neighboring sounds, so context matters. A bidirectional model can exploit both sides when the whole audio sequence is available. A live system must instead respect whatever future context its latency allows. Rendering the predicted facial features into video is another stage, not something an LSTM cell performs by itself.

3.6 LSTM Variants

Most variants change either what a gate can see, how context flows through the data, or the shape of the stored state. Keep those three possibilities separate and the names become easier to remember.

3.6.1 Peephole Connections

Without peepholes, gates see the previous exposed output, not the raw cell state directly. A peephole gives a gate a coordinatewise look at the memory:

$$\begin{aligned}\mathbf{i}_t &= \sigma(\mathbf{s}_{i,t} + \mathbf{p}_i \odot \mathbf{c}_{t-1}),\\ \mathbf{f}_t &= \sigma(\mathbf{s}_{f,t} + \mathbf{p}_f \odot \mathbf{c}_{t-1}),\\ \mathbf{o}_t &= \sigma(\mathbf{s}_{o,t} + \mathbf{p}_o \odot \mathbf{c}_t).\end{aligned}$$

The base pre-activations contain the usual input, recurrent, and bias terms. Each $\mathbf{p}_\alpha$ has I entries; it does not mix all memory coordinates. The input and forget gates use the old cell, while the output gate can use the just-computed new cell. This can help with timing or thresholds, but creates extra feedback and gradient paths.

3.6.2 Bidirectional LSTM

Run one LSTM left to right and another right to left, then concatenate their outputs at matching positions: $\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$. Each direction has its own weights. A tagger can now use both earlier and later words to interpret an ambiguous word.

Bidirectionality is useful when the complete input is available. It is not causal forecasting: the backward direction uses later inputs. Biological sequences also have real orientation; processing both ways is a modelling choice for using context, not a claim that the data have no direction.

3.6.3 Multidimensional LSTM

An image has more than one direction of context. A multidimensional LSTM receives predecessor states along several axes. For a two-axis scan, the memory update can combine a left and an upper cell:

$$\mathbf{c}_{r,s} = \mathbf{i}_{r,s} \odot \mathbf{z}_{r,s} + \mathbf{f}^{\leftarrow}_{r,s} \odot \mathbf{c}_{r,s-1} + \mathbf{f}^{\uparrow}_{r,s} \odot \mathbf{c}_{r-1,s}.$$

The scan order must make both predecessors available, with specified boundary states. Multiple incoming memory paths add context, but also change accumulation and gradient behavior.

3.6.4 Pyramid LSTM

The PyraMiD variant discussed here arranges spatial recurrent computation so that groups of positions can be processed in parallel. Its motivation is to handle multidimensional data, such as volumes, more efficiently than a strictly sequential scan. This use of "pyramid" is about spatial processing directions; it should not be confused with every sequence model that downsamples its time axis.

3.6.5 Stacked LSTM

Depth and time are different axes. In a stack, the hidden output of layer l at time t becomes input to layer $l+1$ at the same time. Every layer also carries its own cell and hidden states along time. This adds several stages of transformation per input position, not just a longer sequence.

3.6.6 Grid LSTM

Grid LSTM extends recurrent processing along multiple axes, which may represent spatial directions or network depth. It maintains LSTM state flows along those directions rather than treating depth as only a feedforward transformation. The useful distinction is that recurrence can be used across *depth as well as time*; a grid is more than a stack with a different drawing.

3.6.7 Convolutional LSTM

For a sequence of images, flattening every image into a vector discards the convenient spatial layout. ConvLSTM keeps states as feature maps and replaces dense input/recurrent maps by learned convolutions. For example,

$$\mathbf{I}_t = \sigma(\mathbf{W}_i * \mathbf{X}_t + \mathbf{R}_i * \mathbf{H}_{t-1} + \mathbf{b}_i), \qquad \mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \mathbf{Z}_t.$$

The other gates and the candidate use analogous convolutions, and $\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t)$. Kernels mix local neighborhoods and channels, while the gates act elementwise on the resulting maps. Shared kernels support translation-equivariant feature processing; they do not make the entire network automatically invariant to translation.

Think of a rainfall radar movie: memory at a location needs nearby past locations because clouds move. Kernel size, layer depth, and repeated recurrent steps determine how far spatial information can travel. This is why ConvLSTM fits precipitation nowcasting and video prediction better than a model that treats each pixel history independently.

3.7 Advanced LSTM Architectures

An LSTM stores its working memory in a fixed-size state. What if a task needs a list of items that grows with the input? The following designs separate the controller from a more explicit memory or let it point directly to input items. The short descriptions here are an orientation, not another set of equations to memorize.

3.7.1 Neural Turing Machine

A Neural Turing Machine couples a neural controller to an external memory that it reads and writes through differentiable attention. The memory can hold task-specific items while the

controller learns how to use them. It was demonstrated on simple algorithmic tasks such as copying and associative recall.[1]

3.7.2 Differentiable Neural Computer

A Differentiable Neural Computer also combines a neural controller with an external memory. Its memory-management mechanisms help organize stored items and their relationships, so the controller can work with structured information rather than squeeze everything into its hidden vector.[2]

3.7.3 Pointer Network

Sometimes the answer should be an *input position*, not a word from a fixed vocabulary. A pointer network turns attention over input positions into its output distribution. For sorting the input list $[8, 2, 5]$, the desired output positions are $[2, 3, 1]$. The number of possible positions grows with the input length. This is different from averaging input features into a context vector: here the pointer distribution itself specifies the output choice.[3]

3.8 Gated Recurrent Unit

A GRU keeps the gating idea with one state $\mathbf{h}_t$, rather than a separate cell state and gated readout. The update gate blends old and new content; the reset gate controls how much previous state is used *when forming the candidate*.

We use the minimal reset-before form here, where the gates depend only on the previous state. This makes its gradients easy to follow:

$$\mathbf{u}_t = \sigma(\mathbf{R}_u^\top \mathbf{h}_{t-1} + \mathbf{b}_u), \qquad \mathbf{r}_t = \sigma(\mathbf{R}_r^\top \mathbf{h}_{t-1} + \mathbf{b}_r), \tag{3.9}$$

$$\mathbf{z}_t = \tanh\Big(\mathbf{W}_z^\top \mathbf{x}_t + \mathbf{R}_z^\top (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_z\Big), \tag{3.10}$$

$$\mathbf{h}_t = (1 - \mathbf{u}_t) \odot \mathbf{h}_{t-1} + \mathbf{u}_t \odot \mathbf{z}_t. \tag{3.11}$$

In this convention, $u = 0$ keeps the old state and $u = 1$ replaces it by the candidate. Some GRU equations swap those roles by calling the complementary quantity the update gate. Always read the state-update equation rather than trusting the name. A more general version adds input maps $\mathbf{W}_u^\top \mathbf{x}_t$ and $\mathbf{W}_r^\top \mathbf{x}_t$ to the gates.

Reset and update do different jobs

Suppose $h_{t-1} = 0.8$, the computed candidate is $z_t = -0.2$, and $u_t = 0.25$. The new state is $0.75(0.8) + 0.25(-0.2) = 0.55$. A small reset gate would have reduced the old state's influence when *computing* z_t. It would not, by itself, have erased the $0.75h_{t-1}$ carry term.

[1] Optional background: Graves et al., Neural Turing Machines.
[2] Optional background: Graves et al., Hybrid computing using a neural network with dynamic external memory.
[3] Optional background: Vinyals et al., Pointer Networks.

3.8.1 Backpropagation for GRU

Let $\mathbf{d}_t$ be the total gradient arriving at $\mathbf{h}_t$, including current and later losses. For the exact variant above, define $\mathbf{v}_t = \mathbf{R}_z \boldsymbol{\delta}_{z,t}$. One backward step is

$$\begin{aligned}
\boldsymbol{\delta}_{z,t} &= \mathbf{d}_t \odot \mathbf{u}_t \odot (1 - \mathbf{z}_t^2), \\
\boldsymbol{\delta}_{u,t} &= \mathbf{d}_t \odot (\mathbf{z}_t - \mathbf{h}_{t-1}) \odot \mathbf{u}_t \odot (1 - \mathbf{u}_t), \\
\boldsymbol{\delta}_{r,t} &= \mathbf{v}_t \odot \mathbf{h}_{t-1} \odot \mathbf{r}_t \odot (1 - \mathbf{r}_t).
\end{aligned}$$

Send this contribution back to the previous hidden state, then add its own local loss gradient:

$$\mathbf{d}_{t-1} \mathrel{+}= \mathbf{d}_t \odot (1 - \mathbf{u}_t) + \mathbf{v}_t \odot \mathbf{r}_t + \mathbf{R}_u \boldsymbol{\delta}_{u,t} + \mathbf{R}_r \boldsymbol{\delta}_{r,t}.$$

The four terms are the direct carry route, the candidate's recurrent route, and the two gate routes. Weight gradients again use outer products. In particular, the sender for $\mathbf{R}_z$ is $\mathbf{r}_t \odot \mathbf{h}_{t-1}$, not just $\mathbf{h}_{t-1}$:

$$\nabla_{\mathbf{R}_z} \mathcal{L} = \sum_t (\mathbf{r}_t \odot \mathbf{h}_{t-1}) \boldsymbol{\delta}_{z,t}^\mathsf{T}, \quad \nabla_{\mathbf{W}_z} \mathcal{L} = \sum_t \mathbf{x}_t \boldsymbol{\delta}_{z,t}^\mathsf{T}.$$

For $\mathbf{R}_u, \mathbf{R}_r$, use $\mathbf{h}_{t-1}$ and their respective deltas. Biases sum deltas. If input maps are added to the gates, their gradients use $\mathbf{x}_t$ as sender. A reset-after variant changes the candidate graph and therefore needs different derivatives.

3.8.2 Usage

For equal hidden width, a three-branch GRU with input-dependent gates uses fewer parameters than a four-branch vanilla LSTM: $3I(D + I + 1)$ versus $4I(D + I + 1)$ under the same single-bias convention. It is a useful simpler model for sequence prediction and encoder-decoder tasks. Fewer parameters do not guarantee better accuracy or lower wall-clock time on every implementation.

The main conceptual difference is memory access. LSTM can retain a cell value while hiding it behind an output gate. A GRU exposes its state directly and couples keeping with updating. Also, with a tanh candidate and a bounded initial state, the GRU's convex-combination update keeps each state coordinate bounded. An additive LSTM cell can represent an unbounded running count in principle, though learning a reliable counter still requires the right gates. Choose based on the task rather than treating one architecture as universally better.

4

Attention

Imagine translating a long sentence after reading it once, with no chance to look back. You would have to keep every useful detail in your head. That is roughly the problem of an encoder-decoder model that passes only one final state to its decoder. Attention gives the decoder something closer to a set of notes: at each output step, it can look up the parts of the input that seem useful *right now*.

The core computation is much less mysterious than the name suggests. Give each available item a relevance score, turn the scores into weights, and combine the items using those weights. The interesting part is that the scoring rule is learned together with the rest of the model.

4.1 Introduction

Attention is not necessarily a single hard choice, and it does not imply human awareness. Most mechanisms here use a smooth mixture: several items can contribute, but some contribute more than others. This makes the operation differentiable, so a task loss can teach the model where looking would help.

4.1.1 Spatial Attention

In image captioning, "dog" may need information from one region while "ball" needs another. An image encoder provides feature vectors $\mathbf{v}_1, \ldots, \mathbf{v}_M$ for different regions. At output step t, attention forms

$$\mathbf{r}_t = \sum_{j=1}^{M} \alpha_{tj} \mathbf{v}_j.$$

The region weights change as the caption is generated. The vector $\mathbf{r}_t$ is the *context* or *glimpse*: what the model extracts from the image for this step. A heatmap can display those weights, but it should not be treated as a complete explanation of the prediction.

A soft crop instead of a hard rectangle. Another spatial mechanism uses a grid of Gaussian filters to read a patch from an image. The model chooses the grid center, spacing, and width of the filters. Changing the center moves the glimpse; changing the spacing changes its coverage; changing the width changes how broadly each sample averages nearby pixels. Because the filters change smoothly, gradients can teach the model where to look. DRAW uses differentiable read/write attention in an iterative image-generation process; related visual-attention models use successive glimpses to recognize cluttered images.

4.1.2 Gates Introduced Temporal Attention

LSTM gates already select information. The write $\mathbf{i}_t \odot \mathbf{z}_t$ gives some candidate coordinates a strong contribution and others almost none. In that broad sense, gating is a form of attention. But an input gate weights *features at one step*; sequence attention typically weights *stored items at different positions*.

For a simple gate score $e_k = \mathbf{w}_k^\top \mathbf{x}$, the identity

$$e_k = \|\mathbf{w}_k\|_2 \, \|\mathbf{x}\|_2 \cos \angle(\mathbf{w}_k, \mathbf{x})$$

shows why both direction and magnitude matter. A large positive score gives $\sigma(e_k)$ near one, a large negative score gives a value near zero, and zero gives one half. The weights learn which directions are useful to detect.

Sigmoid gates and softmax weights are not the same

Independent sigmoid gates can all be high, or all be low. They do not have to sum to one. Softmax attention distributes a total weight of one across the available items, so the items compete. Also, $\mathbf{i} \odot \mathbf{z}$ preserves separate coordinates, while $\sum_j \alpha_j \mathbf{v}_j$ mixes several vectors into one. Both select information, but they do different jobs.

4.1.3 Temporal Attention in Artificial Neural Networks

It helps to name the three stages. A *score* e_j is an unrestricted number measuring a match. A *weight* α_j is obtained by normalizing the scores. A *context vector* is the information retrieved using those weights. For a query $\mathbf{q}$ and a bank of keys and values,

$$e_j = \text{score}(\mathbf{q}, \mathbf{k}_j), \qquad \alpha_j = \frac{e^{e_j}}{\sum_{m=1}^{M} e^{e_m}}, \qquad \mathbf{r} = \sum_{j=1}^{M} \alpha_j \mathbf{v}_j. \tag{4.1}$$

The query says what we are looking for; a key helps decide whether an item matches; a value is what we retrieve from it. Earlier attention models often use the same hidden vector as both key and value. Section 4.3 makes the separation explicit.

Attention with three items: the entire calculation

Suppose the scores are $(\log 2, 0, \log 3)$. Exponentiating gives $(2, 1, 3)$, so softmax produces

$$\boldsymbol{\alpha} = \left(\frac{1}{3}, \frac{1}{6}, \frac{1}{2}\right).$$

Take values $\mathbf{v}_1 = (1, 0)^\top$, $\mathbf{v}_2 = (0, 2)^\top$, and $\mathbf{v}_3 = (2, 2)^\top$. Then

$$\mathbf{r} = \frac{1}{3}\begin{pmatrix}1\\0\end{pmatrix} + \frac{1}{6}\begin{pmatrix}0\\2\end{pmatrix} + \frac{1}{2}\begin{pmatrix}2\\2\end{pmatrix} = \begin{pmatrix}4/3\\4/3\end{pmatrix}.$$

The third item contributes most, but the output is a mixture, not simply the third value. The weights add to one; the components of the context vector need not.

Subtracting the largest score before exponentiating leaves softmax unchanged and helps numerical stability. Adding the same constant to all scores therefore changes nothing. Multiplying all scores by a large positive factor, in contrast, usually makes the distribution sharper. Soft attention keeps a differentiable mixture; choosing an item with argmax is a hard selection and blocks ordinary differentiation through that choice.

4.2 Attention in Sequence-to-Sequence Models

A fixed final encoder state has to anticipate *every* later question the decoder may ask. Attention keeps all encoder states available instead. The decoder can form a new context for each output token. This removes the requirement that every relevant detail cross the single final-state bridge, although each individual context is still a compressed mixture.

Use $j \in \{1, \dots, T\}$ for source positions and $t \in \{1, \dots, S\}$ for output positions. Let $\mathbf{h}_j \in \mathbb{R}^{d_h}$ be an encoder state and $\mathbf{s}_t \in \mathbb{R}^{d_s}$ a decoder state. In this chapter $\mathbf{s}_t$ names the decoder state, not a gate pre-activation, and $\mathbf{r}_t$ names the attention context so it cannot be confused with an LSTM cell state.

A different question at every output step

When translating "the small red boat", one output step may need the noun, another the color, and another grammatical context. A final-state-only decoder has to recover each from the same compressed memory. An attention decoder asks the encoder states again at every step. It is closer to consulting your notes than trying to remember your notes verbatim.

4.2.1 Additive Attention

A bidirectional encoder combines left and right context at each source position, for example $\mathbf{h}_j = [\overrightarrow{\mathbf{h}}_j; \overleftarrow{\mathbf{h}}_j]$. Thus a vector is associated with a word position but need not describe only that word. To decide whether position j helps with the next output, additive attention uses a small neural network:

$$e_{tj} = \mathbf{v}_a^\top \tanh(\mathbf{W}_s \mathbf{s}_{t-1} + \mathbf{W}_h \mathbf{h}_j + \mathbf{b}_a), \tag{4.2}$$

$$\alpha_{tj} = \frac{\exp(e_{tj})}{\sum_{m=1}^{T} \exp(e_{tm})}, \qquad \mathbf{r}_t = \sum_{j=1}^{T} \alpha_{tj} \mathbf{h}_j. \tag{4.3}$$

With scoring width d_a, the shapes are $\mathbf{W}_s : d_a \times d_s$, $\mathbf{W}_h : d_a \times d_h$, and $\mathbf{v}_a, \mathbf{b}_a \in \mathbb{R}^{d_a}$. The projections let encoder and decoder states have different widths. These matrices are written directly as output-by-input maps, so their shapes explicitly specify the multiplication.

The decoder now updates from its previous state, the previous output embedding, and the retrieved context:

$$\mathbf{s}_t = \mathrm{Dec}(\mathbf{s}_{t-1}, \mathrm{emb}(y_{t-1}), \mathbf{r}_t).$$

A readout produces the next-token distribution. Everything is trained jointly from the prediction loss. We do not need manually labeled alignments for the attention weights to learn useful matches.

It is called *additive* because the projected query and encoder state are added *inside the scoring network*. It is not named after the weighted sum that creates the context; other attention mechanisms use that sum too.

How can the model learn where to look?. Let $\mathbf{d} = \nabla_{\mathbf{r}_t} \mathcal{L}$ and hold the value vectors fixed while differentiating the scores. The softmax derivative gives

$$\frac{\partial \mathcal{L}}{\partial e_{tj}} = \alpha_{tj}\, \mathbf{d}^\top (\mathbf{h}_j - \mathbf{r}_t). \tag{4.4}$$

An item's effect is measured relative to the current mixture. If shifting the context towards that item would lower the loss, gradient descent can raise its score. This is how the downstream task

Figure planned: Looking back while decoding

Draw four encoder states along the bottom and two decoder steps above. Give each decoder step its own set of weighted arrows into a context sum. Alongside, show a small source-by-output heatmap with weights normalized over the source axis. Use an illustrative word reorder, not a fabricated measured alignment.

Figure 4.1: Different output steps retrieve different mixtures of the same encoder states.

teaches a scoring network to retrieve useful information. Encoder states also receive gradients through their values and through the scoring network; the equation above isolates just the score route.

4.2.2 Multiplicative Attention and Local Attention

We can change how matches are scored without changing the score–normalize–retrieve recipe. A dot product is a direct compatibility test. A learned bilinear map first changes which dimensions are compared:

Scoring rule	Formula and what to check
Dot product	$e_{tj} = \mathbf{h}_j^\top \mathbf{s}_t$. Both vectors need the same width.
General / bilinear	$e_{tj} = \mathbf{h}_j^\top \mathbf{W}_a \mathbf{s}_t$, with $\mathbf{W}_a \in \mathbb{R}^{d_h \times d_s}$.
Concat / additive	$e_{tj} = \mathbf{v}_a^\top \tanh(\mathbf{W}_a[\mathbf{s}_t; \mathbf{h}_j] + \mathbf{b}_a)$. Splitting the matrix into blocks gives an additive form.

Some decoder designs form a provisional current state $\mathbf{s}_t$ before attention, rather than query with $\mathbf{s}_{t-1}$ as above. Both schedules are possible, but avoid a circular definition where the same state needs a context that has not yet been computed.

Dot and bilinear scores are called multiplicative attention. Dot products are cheap; additive scores use a nonlinear matching network. Whether one performs better depends on the setup, not a universal rule about short and long sequences.

Global versus local. Global attention considers every source position for every output step: TS source-output pairs in total. Local attention considers only a window around a center p_t. It spends less work on distant positions, but it can miss useful information outside that window.

A monotonic scheme uses a center that advances with the output position, such as $p_t = t$. A predictive scheme learns it, for example

$$p_t = T\sigma\Big(\mathbf{v}_p^\top \tanh(\mathbf{W}_p \mathbf{s}_t)\Big).$$

The window is clipped at source boundaries. To prefer positions close to the center, a model can multiply the content weights by a Gaussian factor:

$$\widetilde{\alpha}_{tj} = \alpha_{tj} \exp\left(-\frac{(j - p_t)^2}{2\sigma_p^2}\right), \qquad |j - p_t| \le D_w.$$

A common choice is $\sigma_p = D_w/2$. After this multiplication the weights no longer necessarily sum to one. If a normalized mixture is wanted, divide by their new sum. Equivalently, add the Gaussian log-weight to the scores before a windowed softmax. That renormalized version differs from leaving the multiplied weights unnormalized.

4.2.3 Self-Attention

So far, a decoder has looked at another sequence: the encoder's outputs. In *self-attention*, the states being compared come from the same sequence. The difference is where the information comes from, not whether the score is additive or a dot product.

An LSTM can keep a bank of its past hidden and cell states and read from that bank instead of relying only on the immediately preceding state. At step t, define

$$\mathbf{H}_{<t} = [\mathbf{h}_1, \dots, \mathbf{h}_{t-1}], \qquad \mathbf{C}_{<t} = [\mathbf{c}_1, \dots, \mathbf{c}_{t-1}].$$

One scoring scheme compares stored states with the current input and a previous retrieved hidden state:

$$e_{tj} = \mathbf{v}_a^\mathsf{T} \tanh(\mathbf{W}_h \mathbf{h}_j + \mathbf{W}_x \mathbf{x}_t + \mathbf{W}_r \widetilde{\mathbf{h}}_{t-1}), \qquad j < t.$$

Normalize these $t-1$ scores to $\boldsymbol{\alpha}_t$ and read

$$\widetilde{\mathbf{h}}_t = \mathbf{H}_{<t} \boldsymbol{\alpha}_t, \qquad \widetilde{\mathbf{c}}_t = \mathbf{C}_{<t} \boldsymbol{\alpha}_t.$$

Notice the matching indices: a bank with $t-1$ columns needs a weight vector with $t-1$ entries. A no-forget LSTM-style cell can now compute its gates from $(\mathbf{x}_t, \widetilde{\mathbf{h}}_t)$ and update $\mathbf{c}_t = \widetilde{\mathbf{c}}_t + \mathbf{i}_t \odot \mathbf{z}_t$. At the first step, use a specified initial memory or zero context rather than trying to normalize an empty bank.

This LSTM with a state bank is called a Long Short-Term Memory-Network (LSTMN). It adds a direct route to older states, but it also stores more information and computes more comparisons: attending over every preceding position gives a quadratic number of pairs across a sequence. Attention does not make a recurrent model non-recurrent by itself. Here, the states are still produced sequentially by an LSTM.

Which positions are allowed?. Self-attention may be causal or bidirectional. A causal next-token model must not consult future positions. Before softmax, set disallowed scores to $-\infty$, making their weights zero. Padding positions are masked in the same way. At least one position in a computed row must remain valid. Masking weights *after* softmax without renormalization is not the same operation, because forbidden positions already took a share of the denominator.

4.2.4 Sentence Embedding via Self-Attention

A classifier needs a fixed-size representation even when sentences have different lengths. Instead of taking the final hidden state or averaging all states equally, attention can learn which states to pool. Stack contextual encoder states as $\mathbf{H} \in \mathbb{R}^{d_h \times T}$ and compute

$$\mathbf{e} = \mathbf{v}_a^\mathsf{T} \tanh(\mathbf{W}_a \mathbf{H}) \in \mathbb{R}^{1 \times T}, \qquad \boldsymbol{\alpha} = \operatorname{softmax}(\mathbf{e})^\mathsf{T}, \qquad \mathbf{r} = \mathbf{H}\boldsymbol{\alpha} \in \mathbb{R}^{d_h}.$$

Softmax runs over the T token positions. Here the learned scorer asks the same task-dependent question of each sentence; there is no separate decoder query changing at each generation step.

One pooled vector may mix too many aspects. To create m summaries, replace $\mathbf{v}_a^\mathsf{T}$ by $\mathbf{V}_a \in \mathbb{R}^{m \times d_a}$:

$$\mathbf{E} = \mathbf{V}_a \tanh(\mathbf{W}_a \mathbf{H}) \in \mathbb{R}^{m \times T}, \qquad \mathbf{A} = \operatorname{softmax}_{\text{rows}}(\mathbf{E}), \qquad \mathbf{C} = \mathbf{H}\mathbf{A}^\mathsf{T} \in \mathbb{R}^{d_h \times m}.$$

Each row of $\mathbf{A}$ contains one distribution over tokens; each column of $\mathbf{C}$ is a pooled sentence view. A penalty such as $\left\| \mathbf{A}\mathbf{A}^\mathsf{T} - \mathbf{I}_m \right\|_F^2$ discourages all views from using the same weights. The unsquared norm has the same zero-penalty target but a different gradient scale.

For nonnegative rows summing to one, a unit row norm favors a sharp distribution, while orthogonality favors different positions. Thus this penalty encourages sharpness as well as

diversity; it does not guarantee that the views discover different human-interpretable topics. These multiple pooling views also should not yet be confused with the full multi-head attention block in a Transformer.

4.3 Key-Value Attention

Using one vector both to decide whether an item matches and to supply its content asks it to do two jobs. Key-value attention separates them. A key is optimized for matching; a value is optimized for carrying the useful payload. The query is the current lookup request.

A contact list is a useful analogy

You search using a name, then retrieve a phone number. The search feature and the returned information do not have to be the same thing. In attention, a query is matched against keys, and the resulting weights combine values. Unlike an exact database lookup, this usually returns a soft mixture of several entries.

A recurrent model can split each stored hidden vector into $[\mathbf{k}_j; \mathbf{v}_j]$, or create keys and values with separate learned maps. Their widths need not be equal. With M stored items, use $\mathbf{K} \in \mathbb{R}^{d_k \times M}$, $\mathbf{V} \in \mathbb{R}^{d_v \times M}$, and $\mathbf{q} \in \mathbb{R}^{d_q}$. An additive lookup is

$$\mathbf{e} = \mathbf{w}_a^\mathsf{T} \tanh\Big(\mathbf{W}_k\mathbf{K} + (\mathbf{W}_q\mathbf{q})\mathbf{1}_M^\mathsf{T}\Big) \in \mathbb{R}^{1\times M},$$
$$\boldsymbol{\alpha} = \mathrm{softmax}(\mathbf{e})^\mathsf{T} \in \mathbb{R}^M, \qquad \mathbf{r} = \mathbf{V}\boldsymbol{\alpha} \in \mathbb{R}^{d_v}.$$

Here $\mathbf{W}_k : d_a \times d_k$, $\mathbf{W}_q : d_a \times d_q$, and $\mathbf{w}_a \in \mathbb{R}^{d_a}$. The outer product with ones repeats the projected query across all M keys. The output width comes from the *values*, regardless of the width used for matching.

Figure planned: Queries match keys; weights retrieve values

Show one query comparing against three key cards. Send the resulting weights to three separate value cards and then into a weighted sum. Reuse the weights 1/3, 1/6, 1/2 from the worked example. Label matching space and returned content separately.

Figure 4.2: Keys decide the match; values provide the retrieved information.

In a sliding-window recurrent model, the query can come from the current state and the bank can contain the previous M states. In cross-attention, queries come from one sequence and keys/values from another. In self-attention, all three come from the same sequence. These choices are independent of the score formula.

You now have the ingredients needed for Transformers: queries, keys, values, a score, a softmax over allowed positions, and a weighted sum. The next chapter builds a whole sequence-processing block around that operation, including how to supply position information when recurrence is removed.

5

Transformer

An RNN carries information forward one state at a time. A Transformer gives each token a different way to find context: it can look directly at the other allowed tokens through attention. The interesting part is how a few familiar operations become a useful sequence model when we put them together.

Think of the sentence "The keys beside the bag are missing." To understand "are", the model should connect it to "keys", even though "bag" is closer. Attention provides that connection. The surrounding network learns what to look for, how to combine the retrieved information, and what to predict from it.

5.1 Architecture

The original Transformer is an encoder-decoder model. The encoder reads an input sequence and creates a contextual vector for each input token. The decoder uses those vectors and the output prefix to predict the next output token. Translation is a natural example: read a German sentence, then generate its English translation.

The building block has two main jobs. **Attention mixes information between positions.** A **feed-forward network (FFN)** then transforms the features at each position separately. Residual connections carry the existing representation around each operation; normalization helps control its scale. Stacking blocks repeats this process, giving later layers increasingly processed representations.

An encoder block contains self-attention and an FFN. A decoder block contains masked self-attention, cross-attention to the encoder, and an FFN. Each sublayer gets its own residual connection and normalization. The final encoder output supplies the memory for *every* decoder block; it is not paired only with the decoder block at the same depth.

Three common arrangements. The same ingredients support different tasks. An *encoder-only* model, such as BERT, reads both sides of a token and produces representations for classification or masked-token prediction. A *decoder-only* language model reads a prefix through causal self-attention and predicts its continuation. An *encoder-decoder* model keeps the source and generated sequence separate and connects them through cross-attention.

"Decoder-only" therefore means a causal stack without the encoder or encoder-decoder cross-attention. It does not mean taking every sublayer of the original decoder unchanged. Also, generation can use a decoder-only model for translation: put the source sentence and an instruction into its prefix.

Figure planned: One Transformer, two information flows

Draw a two-block encoder on the left and decoder on the right. Label encoder self-attention, decoder causal self-attention, cross-attention, and position-wise FFN. Show residual paths. Send the final encoder output to both cross-attention blocks. Bottom: source tokens and shifted target tokens. Top: next-token probabilities.

Figure 5.1: The encoder builds a source memory; the decoder reads it while generating an output sequence.

Parallel reading, sequential writing

During training, the complete example is already known. A causal mask lets us compute predictions at all positions in parallel without letting them peek ahead. During generation, the next token does not exist yet: we must choose it before predicting the token after it. Removing recurrence from a block does not remove this dependence between generated tokens.

5.2 Detailed Explanation of the Transformer

From token IDs to vectors

A tokenizer turns text into discrete IDs. Tokens may be words, word pieces, punctuation, or other units; one word need not equal one token. An embedding table $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$ assigns a learned d-dimensional vector to each vocabulary entry. Looking up token u_t selects row u_t of this table. Identical tokens initially get identical token embeddings, but their contextual representations can become very different.

In this chapter, we stack token vectors as **rows**: $\mathbf{X} \in \mathbb{R}^{T \times d}$. This is the transpose of the column-stacked banks in Chapter 4. Individual bold lower-case vectors remain column vectors; row t of $\mathbf{X}$ is $\mathbf{x}_t^\mathsf{T}$. The batch dimension is omitted to keep the equations readable. In code, a batch typically has shape $B \times T \times d$.

For an absolute positional encoding $\mathbf{p}_t$, the first representation is

$$\mathbf{x}_t = \mathbf{e}_{u_t} + \mathbf{p}_t.$$

The original Transformer scales its token embedding by $\sqrt{d}$ before this addition. Other architectures use other conventions. Token embeddings are looked up at the bottom of the stack; later blocks receive the previous block's output rather than looking up the tokens again.

One attention head, with every shape visible

Start with self-attention over T token rows. Three learned projections create queries, keys, and values:

$$\begin{aligned} \mathbf{Q} &= \mathbf{X}\mathbf{W}_Q \in \mathbb{R}^{T\times d_k}, & \mathbf{W}_Q &\in \mathbb{R}^{d\times d_k}, \\ \mathbf{K} &= \mathbf{X}\mathbf{W}_K \in \mathbb{R}^{T\times d_k}, & \mathbf{W}_K &\in \mathbb{R}^{d\times d_k}, \\ \mathbf{V} &= \mathbf{X}\mathbf{W}_V \in \mathbb{R}^{T\times d_v}, & \mathbf{W}_V &\in \mathbb{R}^{d\times d_v}. \end{aligned}$$

Biases are omitted here. Each row of $\mathbf{Q}$ asks a question, each row of $\mathbf{K}$ describes what an entry matches, and each row of $\mathbf{V}$ supplies its content. These are learned features, not literal words or manually assigned labels.

Scaled dot-product attention

For an additive mask $\mathbf{M} \in \mathbb{R}^{T\times T}$,

$$\mathbf{A} = \operatorname{softmax}_{\text{row}}\left(\frac{\mathbf{Q}\mathbf{K}^{\mathsf{T}}}{\sqrt{d_k}} + \mathbf{M}\right), \qquad \mathbf{O} = \mathbf{A}\mathbf{V} \in \mathbb{R}^{T\times d_v}.$$

Row i gives the weights used by query i over key positions j. Thus

$$a_{ij} = \frac{\exp(s_{ij})}{\sum_{\ell \text{ allowed for } i} \exp(s_{i\ell})}, \qquad \mathbf{o}_i = \sum_{j \text{ allowed for } i} a_{ij}\mathbf{v}_j, \quad s_{ij} = \frac{\mathbf{q}_i^{\mathsf{T}}\mathbf{k}_j}{\sqrt{d_k}},$$

with $a_{ij} = 0$ at masked positions. Every nonempty row sums to one.

The shape chain is worth checking once: $(T \times d_k)(d_k \times T)$ gives $T \times T$ scores; multiplying the resulting weights by $T \times d_v$ values gives $T \times d_v$ outputs. The output width is set by the values, not by the number of keys.

Why divide by $\sqrt{d_k}$?. Suppose, as a rough scale argument, query and key coordinates are independent, centered, and have variance one. Their dot product is a sum of d_k independent terms of variance one, so its variance is d_k. Dividing by $\sqrt{d_k}$ brings that variance back to one. Without it, wider heads tend to create more extreme scores, and softmax can become almost one-hot before useful matching has been learned. This is a scale argument, not a claim that trained features remain independent or Gaussian.

A complete lookup with two features

Let $d_k = 2$, $\mathbf{q} = (\sqrt{2}, 0)^{\mathsf{T}}$, and choose key rows and value rows

$$\mathbf{K} = \begin{pmatrix} \ln 2 & 0 \\ 0 & 1 \\ \ln 3 & 1 \end{pmatrix}, \qquad \mathbf{V} = \begin{pmatrix} 2 & 0 \\ 0 & 6 \\ 4 & 2 \end{pmatrix}.$$

The scaled scores are $(\ln 2, 0, \ln 3)$. Exponentiation gives $(2, 1, 3)$, so the weights are $(1/3, 1/6, 1/2)$. The output is

$$\mathbf{o} = \tfrac{1}{3}(2,0)^{\mathsf{T}} + \tfrac{1}{6}(0,6)^{\mathsf{T}} + \tfrac{1}{2}(4,2)^{\mathsf{T}} = (8/3, 2)^{\mathsf{T}}.$$

If position 3 is forbidden, remove its mass *before normalizing*: the weights become $(2/3, 1/3, 0)$ and the output becomes $(4/3, 2)^{\mathsf{T}}$. Keeping the old allowed weights would incorrectly shrink their total to $1/2$.

Masking: who is allowed to see whom?

A causal language model lets position i attend to positions $j \le i$. Its mask is

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases} \qquad \mathbf{M} = \begin{pmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{pmatrix} \quad (T = 3).$$

The diagonal is allowed because the output at position i predicts the *next* token. Padding masks independently remove dummy keys used to make batch lengths match. Loss masking removes padding targets from the objective; it does not replace the attention mask. Packed independent documents can also be prevented from attending across their boundaries.

The mask is added to *scores*, before softmax. Multiplying a score by zero does not forbid it, since $\exp(0) = 1$. An entirely masked row has no valid probability distribution. Implementations must avoid such rows or handle them explicitly, especially for padding.

Figure planned: Read the causal triangle

Draw three small score grids with query rows and key columns: full encoder attention, causal decoder attention, and causal attention with a short sliding window. Highlight allowed diagonal entries. Under the causal grid align input BOS / I / study with targets I / study / EOS.

Figure 5.2: The mask controls information access; shifting the targets makes next-token prediction possible.

Multi-head attention: several lookups, then a learned mix

One head gives each token one weighted combination of values. With H heads, each head learns its own projections and its own attention weights. A head might help resolve a reference while another emphasizes nearby syntax, but those roles are not assigned in advance and need not be neatly interpretable.

For head h, compute $\mathbf{O}^{(h)} = \mathrm{Attention}(\mathbf{Q}^{(h)}, \mathbf{K}^{(h)}, \mathbf{V}^{(h)})$. Concatenate along the feature axis and project back to the model width:

$$\mathrm{MHA}(\mathbf{X}) = [\mathbf{O}^{(1)} \mid \cdots \mid \mathbf{O}^{(H)}]\mathbf{W}_O, \qquad \mathbf{W}_O \in \mathbb{R}^{Hd_v \times d}.$$

The concatenation has shape $T \times Hd_v$ and the result has shape $T \times d$, so a residual addition with $\mathbf{X}$ is possible. The heads are not averaged, and $\mathbf{W}_O$ can mix their features.

For $d = 512$, $H = 8$, and $d_k = d_v = 64$, one head projects 512 input features to 64 query/key/value features. Concatenating eight outputs restores 512 features. In code, a fused projection is usually reshaped into $B \times H \times T \times d_k$; the head axis is distinct from the token axis. The familiar choice $Hd_v = d$ is convenient, not a mathematical requirement.

Cross-attention uses a different memory

Let decoder representations be $\mathbf{D} \in \mathbb{R}^{T_y \times d}$ and final encoder representations be $\mathbf{Z} \in \mathbb{R}^{T_x \times d}$. A cross-attention head uses

$$\mathbf{Q} = \mathbf{D}\mathbf{W}_Q, \qquad \mathbf{K} = \mathbf{Z}\mathbf{W}_K, \qquad \mathbf{V} = \mathbf{Z}\mathbf{W}_V.$$

Its attention matrix has shape $T_y \times T_x$: each output position asks where to look in the input. The decoder's self-attention is causal. Its cross-attention usually sees the whole source sentence, because that sentence is already available; source padding still needs masking. Self- versus cross-attention tells us where Q, K, V come from, while causal versus full attention tells us which pairs are permitted.

Position: why a bag of tokens is not enough

Without positions or an order-dependent mask, self-attention cannot distinguish different orders beyond moving its outputs with the inputs. For a permutation matrix $\mathbf{P}$,

$$\mathrm{SA}(\mathbf{P}\mathbf{X}) = \mathbf{P}\,\mathrm{SA}(\mathbf{X}).$$

This property is *permutation equivariance*: rearrange input rows and the outputs rearrange in the same way. For one fixed query, rearranging matched key-value pairs leaves its retrieval unchanged. That latter property is invariance. The distinction matters when explaining why an unpositioned encoder treats a sequence like a set.

Absolute positions. Learned positional embeddings give each position its own trainable vector. Sinusoidal encodings instead use fixed frequencies. For even d, position t, and $r = 0, \ldots, d/2 - 1$,

$$p_{t,2r} = \sin(t/10000^{2r/d}), \qquad p_{t,2r+1} = \cos(t/10000^{2r/d}).$$

Fast and slow oscillations together provide a position signature. A shift changes each sine/cosine pair by a rotation, giving the model a structured way to work with relative offsets. Being able to compute the encoding beyond the training length does not guarantee that the model will work well there.

Rotary positions (RoPE). RoPE puts position into the attention comparison by rotating pairs of query and key coordinates. For a two-dimensional pair, define

$$\mathbf{R}(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}, \quad \widetilde{\mathbf{q}}_i = \mathbf{R}(i\omega)\mathbf{q}_i, \quad \widetilde{\mathbf{k}}_j = \mathbf{R}(j\omega)\mathbf{k}_j.$$

Then

$$\widetilde{\mathbf{q}}_i^{\mathsf{T}}\widetilde{\mathbf{k}}_j = \mathbf{q}_i^{\mathsf{T}}\mathbf{R}((j-i)\omega)\mathbf{k}_j.$$

The rotation in the score depends on the relative offset $j - i$. Different coordinate pairs use different frequencies. Values are usually not rotated. RoPE therefore changes how positions match; it does not add a position vector to every value. Context-extension methods adjust this positional machinery and/or the training procedure, but a larger configured context is not a guarantee of reliable recall.

Order can enter in more than one place

Absolute embeddings attach a position label to the token representation. RoPE changes the query-key comparison using position. A causal mask supplies direction by giving earlier and later positions different sets of visible tokens. These mechanisms are different, and a model can combine them. Removing explicit position embeddings does not remove a causal mask's asymmetry.

Feed-forward layers: process each token's new information

After attention has mixed tokens, the FFN applies the same small neural network to each row:

$$\mathrm{FFN}(\mathbf{X}) = \sigma(\mathbf{X}\mathbf{W}_1 + \mathbf{1}\mathbf{b}_1^\mathsf{T})\mathbf{W}_2 + \mathbf{1}\mathbf{b}_2^\mathsf{T},$$

where $\mathbf{W}_1 \in \mathbb{R}^{d\times d_{\text{ff}}}$ and $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}}\times d}$. The original uses ReLU and $d_{\text{ff}} = 4d$. Here σ denotes the chosen activation, not necessarily a sigmoid. The hidden layer expands the representation, applies a nonlinearity, and projects it back. Parameters are shared across token positions, but ordinarily not across depth.

A gated FFN adds a second projection. Ignoring biases, a standard SwiGLU form is

$$\mathrm{SwiGLU}(\mathbf{X}) = \big[\mathrm{SiLU}(\mathbf{X}\mathbf{W}_g) \odot (\mathbf{X}\mathbf{W}_u)\big]\mathbf{W}_d, \qquad \mathrm{SiLU}(a) = a\ \mathrm{sigmoid}(a).$$

Both up-projections have shape $d \times m$ and the down-projection has shape $m \times d$. One branch controls how much of the other passes through. Unlike an LSTM sigmoid gate, the SiLU branch is not restricted to $[0, 1]$. GELU, used by GPT-2, is $a\Phi(a)$, where Φ is the standard normal CDF.

A plain FFN has about $2dd_{\text{ff}}$ matrix parameters; a gated one has about $3dm$. Matching that parameter budget gives $m \approx 2d_{\text{ff}}/3$. There is no universal rule that every gated hidden width must be half the old width; real configurations choose their own dimensions.

Residual paths and normalization

A sublayer should be able to improve a representation without rebuilding it from scratch. A residual connection implements $\mathbf{x} + F(\mathbf{x})$: keep the current vector and add an update. Its derivative includes an identity term, $\mathbf{I} + J_F$, giving gradients a direct route across depth. That helps optimization but does not guarantee perfect gradients through an entire network.

Layer normalization acts across the d features *of one token*, not across tokens or a batch. With learned scale $\boldsymbol{\gamma}$, shift $\boldsymbol{\beta}$, and small $\epsilon > 0$,

$$\mathrm{LN}(\mathbf{x}) = \boldsymbol{\gamma} \odot \frac{\mathbf{x} - \mu\mathbf{1}}{\sqrt{v + \epsilon}} + \boldsymbol{\beta}, \quad \mu = \frac{1}{d}\sum_i x_i, \quad v = \frac{1}{d}\sum_i (x_i - \mu)^2.$$

RMSNorm rescales without subtracting the mean, usually without a learned shift:

$$\mathrm{RMSNorm}(\mathbf{x}) = \boldsymbol{\gamma} \odot \frac{\mathbf{x}}{\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon}}.$$

For $\mathbf{x} = (1, 3)^\mathsf{T}$, unit scale, zero shift, and ignoring ϵ, LN gives $(-1, 1)^\mathsf{T}$; RMSNorm gives $(1, 3)^\mathsf{T}/\sqrt{5}$. Both control scale, but only LN centers the features.

The placement changes the computation. With N a normalization and F attention or an FFN, three useful arrangements are

$$\begin{aligned} \text{post-norm:} \quad & \mathbf{y} = N(\mathbf{x} + F(\mathbf{x})), \\ \text{pre-norm:} \quad & \mathbf{y} = \mathbf{x} + F(N(\mathbf{x})), \\ \text{branch-output norm:} \quad & \mathbf{y} = \mathbf{x} + N(F(\mathbf{x})). \end{aligned}$$

The original Transformer uses post-norm. Pre-norm leaves the residual route outside normalization, which often makes deep stacks easier to optimize; a final normalization is commonly applied after the stack. Branch-output normalization also keeps the skip path outside the norm but normalizes a different quantity. We will meet this in OLMo 2. These placements are not interchangeable, and pre-norm does not make learning-rate warm-up universally unnecessary.

A complete original encoder block. Omitting dropout for readability, its two steps are

$$\mathbf{U} = \mathrm{LN}(\mathbf{X} + \mathrm{MHA}(\mathbf{X})), \qquad \mathbf{Y} = \mathrm{LN}(\mathbf{U} + \mathrm{FFN}(\mathbf{U})).$$

A decoder inserts a residual cross-attention sublayer between causal self-attention and the FFN. The original architecture also uses dropout and a warm-up/decay learning-rate schedule. Such training choices belong to the specific setup; they are not part of the definition of attention.

Training: predict the next token at every position

The last decoder representation is projected to vocabulary logits. With $\mathbf{H} \in \mathbb{R}^{T \times d}$,

$$\mathbf{L} = \mathbf{H}\mathbf{W}_{\text{vocab}} + \mathbf{1}\mathbf{b}_{\text{vocab}}^{\mathsf{T}} \in \mathbb{R}^{T \times |\mathcal{V}|}, \qquad \mathbf{P} = \mathrm{softmax}_{\text{row}}(\mathbf{L}).$$

This softmax is over *vocabulary entries*. Attention's softmax was over *key positions*. Confusing those two axes makes the whole model harder to follow. Weight tying can reuse the embedding table via $\mathbf{W}_{\text{vocab}} = \mathbf{E}^{\mathsf{T}}$ when dimensions match.

For an output sequence $y_1, \ldots, y_T$, the conditional language-model objective is

$$p(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^{T} p(y_t \mid y_{<t}, \mathbf{x}), \qquad \mathcal{L} = -\sum_{t=1}^{T} \log p(y_t \mid y_{<t}, \mathbf{x}).$$

Here $\mathbf{x}$ is the encoder input; omit it for an unconditional decoder-only language model, or include a prompt in the prefix. A practical loss is often averaged over valid target tokens.

What exactly is shifted?

For the target "I study" followed by an end token, feed the decoder

input: [`BOS`, `I`, `study`], target: [`I`, `study`, `EOS`].

The representation at `I` may use `BOS` and `I`, and predicts `study`. Although `study` appears elsewhere in the training tensor, the causal mask hides it from this position. Without the mask, the model could read its own answer.

This is teacher forcing: during training, prefixes contain the true earlier tokens. Backpropagation updates the embeddings, projections, FFNs, and other learned parameters together. During generation, prefixes contain the model's chosen tokens, so a mistaken continuation can affect later predictions.

Generation, sampling, and the KV cache

Generation repeatedly computes a next-token distribution, chooses a token, and appends it. It stops at an end token or a length limit. *Greedy* decoding chooses the largest probability. *Temperature* rescales logits before softmax, $p_i \propto \exp(\ell_i/\tau)$ with $\tau > 0$: smaller τ sharpens the distribution; larger τ flattens it.

Top-k sampling keeps the k largest probabilities and renormalizes them. *Top-p* (nucleus) sampling sorts probabilities and keeps the smallest leading set whose cumulative mass is *at least* p, then renormalizes. For probabilities $(0.50, 0.25, 0.15, 0.10)$, top-p with $p = 0.8$ keeps the first three entries, whose mass is 0.90. Sampling changes output diversity; it does not guarantee better answers.

Why cache keys and values?. In a causal model, adding a new token does not change the old tokens' representations: they still cannot see the future. At each layer we can keep their keys and values and compute only the new token's projections. Its query attends to the cached history plus the current entry. Old queries are not needed for this next lookup.

Processing the initial prompt is called *prefill*. Subsequent one-token steps are *decode*. The cache saves recomputation, but a global attention head still reads a history that grows with length. Each layer needs its own cache because it operates on different representations. RoPE positions must also remain consistent when appending entries.

For L layers, batch size B, history length T, H_{kv} key-value heads, head width d_h for both keys and values, and b bytes per element,

$$\text{KV bytes} = 2LBTH_{\text{kv}}d_h b.$$

The factor two counts keys *and* values. This excludes model weights, temporary buffers, and implementation overhead.

The cache can be surprisingly large

Take $L = 32$, $B = 1$, $T = 8192$, $d_h = 128$, and two-byte elements. With 32 KV heads the cache is

$$2 \cdot 32 \cdot 8192 \cdot 32 \cdot 128 \cdot 2 = 4{,}294{,}967{,}296 \text{ bytes} = 4 \text{ GiB}.$$

Sharing keys and values so that only 8 KV heads remain reduces this to 1 GiB. The number of query heads can stay at 32. This is the motivation for grouped-query attention, not a reduction in the number of generated tokens.

Where the quadratic cost comes from

With fixed head widths, full attention compares all T queries with all T keys. One head needs $\mathcal{O}(T^2(d_k + d_v))$ work for scoring and retrieval. Materializing its weights requires $\mathcal{O}(T^2)$ entries. With H heads, that is HT^2 attention entries per example; doubling context multiplies this part by four.

The projections and FFNs also cost work, roughly $\mathcal{O}(Td^2)$ and $\mathcal{O}(Tdd_{\text{ff}})$ per layer for conventional widths. Quadratic attention is therefore not automatically the largest cost at every length. At one cached decode step, attention costs $\mathcal{O}(T(d_k + d_v))$ per query head; the sum over many growing-prefix steps is still quadratic in the final length.

Memory-efficient exact attention implementations can avoid storing the full score matrix by computing tiles and accumulating the softmax result. They preserve the dense attention calculation while reducing memory traffic and intermediate storage. This does not turn the

all-pairs arithmetic into linear attention. A different kernel or a restricted attention pattern changes a different part of the problem.

5.3 Bidirectional Encoder Representations from Transformers (BERT)

BERT uses an encoder stack to build representations from both left and right context. In "She put the money in the [MASK]", the surrounding words help recover "bank". A causal next-token model cannot use words after the token being predicted. BERT can, because masked-token prediction supplies a different training task.

Input representation. BERT adds token, position, and segment embeddings. For two segments, the sequence has the form

$$[\texttt{CLS}] \quad A \quad [\texttt{SEP}] \quad B \quad [\texttt{SEP}].$$

Segment embeddings distinguish A from B; position embeddings mark locations in the combined sequence. The final `CLS` representation can feed a sequence-level classifier. A `SEP` token separates segments or marks the end. Special tokens get contextual representations just like other tokens.

Masked language modeling (MLM). Select a subset $\mathcal{M}$ of token positions and corrupt their inputs. Train the model to recover the original tokens at those positions:

$$\mathcal{L}_{\text{MLM}} = -\sum_{t \in \mathcal{M}} \log p(u_t \mid \tilde{\mathbf{u}}).$$

The original BERT recipe selects 15% of positions. Of those selected positions, 80% become `MASK`, 10% are replaced by a random token, and 10% stay unchanged. All selected positions contribute to the prediction loss. This reduces reliance on seeing a special mask token, which is absent from many downstream inputs. The label is the *original* token, including when its input has been replaced.

Next sentence prediction (NSP). The original second pretraining task asks whether segment B follows segment A in the source text. Positive and randomly paired negative examples supply the binary labels automatically. A classifier on the sequence representation predicts the label, and its cross-entropy is added to the MLM loss. NSP predicts a relation between two supplied segments; it does not generate the next sentence.

Fine-tuning. Pretraining learns reusable representations. A task head then maps them to the desired output, and the original BERT fine-tuning setup updates the encoder as well as the head. Sentiment classification can use `CLS`; token labeling uses each token's representation; extractive question answering predicts start and end positions in a supplied passage. That last task selects a span rather than freely generating an answer.

BERT and a causal language model answer different questions

MLM asks "What belongs in this corrupted position, given the visible context?" Next-token prediction asks "What comes next, given only the prefix?" Original BERT's MLM objective does not directly provide the left-to-right sequence likelihood above, and its bidirectional encoder is not a drop-in autoregressive generator.

5.4 LLM Architectures

The names can make this topic look like ten unrelated architectures. Most of the differences are easier to remember as choices along a few axes: how positions enter, where normalization sits, how the KV cache is stored, whether attention is local, and whether the FFN is dense or routed. The following models are concrete examples of those choices, not a ranking of current products.

Shared vocabulary for the comparisons. In *multi-head attention* (MHA), each query head has its own key and value head. In *multi-query attention* (MQA), all query heads share one KV head. *Grouped-query attention* (GQA) sits between them: H_q query heads share H_{kv} KV heads in groups. Each query still gets its own attention weights. GQA reduces stored KV features and their projection parameters; it does not remove comparisons with the history or force different queries to retrieve the same mixture.

Figure planned: What gets shared in the cache?

Use eight query-head boxes in each of three columns. Connect them to eight KV pairs for MHA, two KV pairs for GQA, and one pair for MQA. Keep the eight query outputs visible in all cases. Label cache size as proportional to the number of KV heads.

Figure 5.3: Sharing keys and values reduces cache storage while retaining separate query heads.

5.4.1 GPT-2

GPT-2 is a useful baseline: a decoder-only stack with causal MHA, learned absolute positional embeddings, pre-norm LayerNorm, and dense GELU FFNs. A final normalization and vocabulary projection produce next-token probabilities. Its training objective is next-token prediction, and the same model can continue many kinds of text by conditioning on different prefixes.

The key distinction from the original encoder-decoder Transformer is the information flow: one causal stream, without an encoder memory or cross-attention. A prompt is simply an initial part of that stream. The later architectures change components inside this overall pattern. An instruction-tuned assistant additionally learns how to respond to instructions; a next-token pretraining objective alone does not specify helpful conversational behavior.

5.4.2 DeepSeek V3/R1

Two ideas distinguish this family: compressed attention memory and sparsely activated FFNs. V3 supplies the base architecture; R1's reasoning training changes the learning procedure and resulting behavior rather than introducing a new meaning of Q, K, and V.

Multi-head latent attention (MLA). Instead of caching separate full-width keys and values for all heads, compress a token's representation to a latent vector $\mathbf{c}_t = \mathbf{W}_{\text{down}}\mathbf{x}_t$. Head-specific maps can recover content keys and values from it:

$$\mathbf{k}_t^{C,h} = \mathbf{U}_{K,h}\mathbf{c}_t, \qquad \mathbf{v}_t^{h} = \mathbf{U}_{V,h}\mathbf{c}_t.$$

For content scores, $\mathbf{q}^\top \mathbf{U}_{K,h}\mathbf{c}_t = (\mathbf{U}_{K,h}^\top \mathbf{q})^\top \mathbf{c}_t$. Similarly, a weighted value sum can apply $\mathbf{U}_{V,h}$ after accumulating latent vectors. These rearrangements explain how inference can use the

compressed cache without materializing all expanded past entries. DeepSeek's MLA also uses a separate positional key component for RoPE, so "cache only $\mathbf{c}_t$" is an incomplete description of the full mechanism.

Mixture-of-experts (MoE). Replace a dense FFN with multiple expert FFNs and a router. For input $\mathbf{x}$, the router picks a small index set $\mathcal{I}(\mathbf{x})$ and weights their outputs:

$$F_{\text{MoE}}(\mathbf{x}) = F_{\text{shared}}(\mathbf{x}) + \sum_{e \in \mathcal{I}(\mathbf{x})} g_e(\mathbf{x}) F_e(\mathbf{x}).$$

The shared term is optional. The g_e and selection/normalization rules depend on the architecture. Routing is per token and layer, so different tokens in one batch can activate different experts. These experts are FFN modules, not complete independent language models, and their specialization need not match human subject labels.

For V3, the first three blocks have dense FFNs; later MoE blocks use 256 routed experts, of which eight are selected per token, plus one always-active shared expert. The model has about 671B total and 37B active parameters per token.[1] Count "eight routed plus one shared", not nine selected from 256.

A large library, a small reading list

MoE keeps a large collection of weights but consults only part of it for one token. That can increase capacity without evaluating every expert. The whole collection still needs storage, and routing can add communication and load-balancing costs. Active parameter count helps describe computation; it is not the model's total memory footprint or a direct latency measurement.

5.4.3 OLMo 2

OLMo 2 illustrates that normalization details can matter even when the large-scale architecture looks familiar. It uses RMSNorm on attention and FFN outputs *inside* the residual branch:

$$\mathbf{y} = \mathbf{x} + \text{RMSNorm}(F(\mathbf{x})).$$

That is the branch-output arrangement introduced above. It differs from normalizing the residual sum $\mathbf{x} + F(\mathbf{x})$.

It also applies RMSNorm to projected queries and keys before RoPE and score computation. This *QK normalization* controls query/key magnitudes and helps stabilize attention logits; it is separate from normalizing the residual stream. The exact normalization axes belong to the implementation, so "QK-norm" alone does not specify an entire attention layer. OLMo 2 additionally emphasizes releasing training resources for reproducible research.[2] A smoother training curve supports a stability claim for that setup, not a proof that one normalization placement always wins.

5.4.4 Gemma 3

Gemma 3 makes local versus global attention easy to see. Its attention pattern combines five sliding-window layers with a global layer, using a local window of 1024 tokens in the compared architecture. In a causal model, a local window looks backward and includes the current position; it does not peek at future tokens.

[1] Architecture details checked against the DeepSeek-V3 technical report.
[2] See the OLMo 2 architecture and training description.

A width-w window requires roughly $\mathcal{O}(Tw)$ query-key comparisons per layer rather than $\mathcal{O}(T^2)$. Local layers can discard KV entries outside their window during decoding. Global layers still need the full history, so mixing the two saves memory without making the *whole* model independent of context length. Several local layers can transmit information farther than a single window, but that is different from directly retrieving any old token in one layer.

Gemma 3 also uses GQA and normalization around sublayers. The larger original Gemma 3 variants accept images and text through a vision component and a language model; the 1B variant is text-only. Do not transfer the capabilities of every Gemma-family model to every size.[3] The related Gemma 3n introduces per-layer embeddings and selective loading/caching strategies for device efficiency. It is a distinct design; those memory-saving features should not be assumed for ordinary Gemma 3.

5.4.5 Mistral Small 3.1

Mistral Small 3.1 is a dense model whose text backbone uses GQA and full causal attention rather than Gemma 3's interleaved sliding-window pattern. Its design illustrates a broader trade-off: fewer layers, different widths, and a smaller KV cache can change runtime in different ways. A custom tokenizer also changes how much text fits into a given number of tokens.

Do not infer latency directly from parameter count or an attention mask. A deeper but narrower model can have fewer parameters and still require more sequential layer computations. A local mask reduces the mathematical workload only if the implementation actually exploits its sparsity. Benchmark comparisons are meaningful only with the task, context, precision, batch size, and hardware held in view.

5.4.6 Llama 4

Llama 4 brings together multimodal input and MoE. The Maverick configuration is a useful contrast to DeepSeek V3: it uses GQA rather than MLA and alternates dense and MoE blocks. Its routed experts are fewer and larger; it activates one routed expert alongside a shared expert in an MoE block. DeepSeek V3 instead combines several smaller routed experts.

The learning point is the granularity of conditional computation. Many small experts let a router combine several pieces; fewer large experts make each chosen branch more substantial. Neither arrangement is automatically better. Keep the specific variant in mind: "Llama 4" is a family name, not one fixed expert count or context configuration.

5.4.7 Qwen3

Qwen3 includes both dense and MoE variants. The names 30B-A3B and 235B-A22B distinguish approximate *total* parameters from parameters *active per token*. The compared MoE designs omit an always-on shared expert, unlike DeepSeek V3. That is an architectural choice about where to place capacity, not an absence of shared attention or shared embeddings.

Qwen3 also provides examples of GQA, rotary positions, gated FFNs, and QK normalization. At a similar parameter budget, changing depth and width changes the cache and execution pattern. A smaller model can therefore use less memory without necessarily generating more tokens per second in every setup. Reasoning/non-reasoning modes concern training and generation behavior; they do not imply swapping the entire Transformer architecture at runtime.

[3] Variant scope: Gemma 3 model card.

5.4.8 SmolLM3

SmolLM3 is a compact decoder with GQA and a hybrid positional scheme: every fourth layer omits RoPE, while the other layers retain it.[4] This is the relevant meaning of *NoPE* here. Calling the entire model position-free would miss both the remaining rotary layers and the causal mask.

The motivation is to improve long-context behavior without imposing the same explicit positional transformation in every layer. In a NoPE layer, attention can still use representations that were position-aware in earlier layers. The causal visibility pattern also provides order information. Removing positional embeddings is therefore neither equivalent to shuffling the input nor a general guarantee of length extrapolation.

5.4.9 Kimi 2

The model discussed here is **Kimi K2**. It follows the MLA-plus-MoE direction of DeepSeek V3, with more routed experts and fewer attention heads in the compared configuration. K2 has roughly 1T total parameters and 32B active per token: another clear example of why total and active counts answer different questions.

Its training uses Muon-based optimization with the MuonClip stability modification.[5] This changes parameter updates during training, not the attention readout at inference. A smooth training loss is useful evidence about optimization; it is not by itself evidence of better reasoning, factual accuracy, or long-context recall.

5.4.10 OpenAI GPT OSS

The gpt-oss-20b and gpt-oss-120b architectures combine pre-RMSNorm, RoPE, GQA, and MoE with a modified SwiGLU activation. They alternate local and global causal attention. Each token selects four experts; the smaller model has 32 experts per MoE block and the larger has 128. Their expert weights use MXFP4 quantization, reducing storage compared with higher-precision weights. Quantization trades numerical precision for memory and bandwidth; runtime also depends on kernel support and cache size.[6]

A learned place for unused attention mass. Ordinary softmax forces a query to distribute all its attention mass over available tokens. A learned extra logit b_h per head can act like a zero-value destination:

$$\mathbf{o}_i = \frac{\sum_{j \in \mathcal{J}_i} \exp(s_{ij}) \mathbf{v}_j}{\exp(b_h) + \sum_{j \in \mathcal{J}_i} \exp(s_{ij})}.$$

Here $\mathcal{J}_i$ is the allowed set. The extra term appears only in the denominator, letting attention reduce its contribution when no token is useful. It is not a bias simply added equally to all real-token scores; such a common shift would cancel under ordinary softmax. This also clarifies the connection to attention sinks, where real tokens absorb attention mass without useful content.

Architecture versus test-time computation. A reasoning model can spend more generated tokens on intermediate work before answering. This increases computation at inference without increasing its parameter count. Longer generation is an opportunity to improve an answer, not a mathematical guarantee. Evaluation needs the task and token budget; a result on one math benchmark is not a universal ranking.

[4] See the SmolLM3 architecture description.
[5] Variant and training details: Kimi K2 technical overview.
[6] Architecture and quantization details: gpt-oss model card.

6

Linear Attention: Outer Product Memories

Softmax attention keeps past keys and values so that each new query can inspect them. What if we combine that history into one fixed-size memory matrix instead? The payoff is a recurrent update whose storage does not grow with sequence length. The price is that individual entries can interfere, and the resulting lookup is generally different from softmax attention.

6.1 Outer-Product Memory and Retrieval

Use column vectors again: $\mathbf{k}_t, \mathbf{q}_t \in \mathbb{R}^{d_k}$ and $\mathbf{v}_t \in \mathbb{R}^{d_v}$. An outer product $\mathbf{v}_t \mathbf{k}_t^\mathsf{T}$ has shape $d_v \times d_k$. It writes an association from a key direction to a value direction. Start with $\mathbf{S}_0 = \mathbf{0}$ and add each association:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\mathsf{T}, \qquad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t.$$

Here we read *after* writing, so the current item is included, matching a causal mask with an allowed diagonal. Reading from $\mathbf{S}_{t-1}$ would instead restrict the lookup to earlier items.

Expand the recurrence and distribute the matrix-vector product:

$$\mathbf{o}_t = \left(\sum_{j=1}^{t} \mathbf{v}_j \mathbf{k}_j^\mathsf{T} \right) \mathbf{q}_t = \sum_{j=1}^{t} \mathbf{v}_j (\mathbf{k}_j^\mathsf{T} \mathbf{q}_t).$$

This is an attention-like weighted sum with raw dot-product weights. Those weights can be negative and do not have to sum to one. Adding more writes can increase the output magnitude. We have not secretly computed softmax.

A tiny associative memory

Write keys $\mathbf{k}_1 = (1, 0)^\mathsf{T}$, $\mathbf{k}_2 = (0, 1)^\mathsf{T}$ with values $\mathbf{v}_1 = (2, 1)^\mathsf{T}$, $\mathbf{v}_2 = (-1, 3)^\mathsf{T}$. Then

$$\mathbf{S}_2 = \begin{pmatrix} 2 & 0 \\ 1 & 0 \end{pmatrix} + \begin{pmatrix} 0 & -1 \\ 0 & 3 \end{pmatrix} = \begin{pmatrix} 2 & -1 \\ 1 & 3 \end{pmatrix}.$$

Query $(1, 0)^\mathsf{T}$ retrieves $(2, 1)^\mathsf{T}$ exactly. Query $(0, 1)^\mathsf{T}$ retrieves $(-1, 3)^\mathsf{T}$. Query $(1, 1)^\mathsf{T}$ retrieves their sum $(1, 4)^\mathsf{T}$. Exact individual recall works here because the keys are orthonormal. With overlapping keys, the other writes also contribute.

Figure planned: Store associations in a matrix

Show the two outer products from the example filling two columns of a 2-by-2 matrix. Multiply by the first basis query to light up only the first column. Add a small inset with overlapping keys to show cross-talk.

Figure 6.1: Outer-product memory stores key-value associations; similar key directions can interfere.

Kernel features and normalization

To obtain nonnegative normalized weights, replace the dot product with a feature kernel

$$\kappa(\mathbf{q}, \mathbf{k}) = \phi(\mathbf{q})^{\mathsf{T}} \phi(\mathbf{k}), \qquad \phi : \mathbb{R}^{d_k} \longrightarrow \mathbb{R}^r.$$

Choose features with nonnegative inner products, for example elementwise $\phi(\mathbf{x}) = \mathrm{ELU}(\mathbf{x}) + 1$, where $\mathrm{ELU}(x) = x$ for $x > 0$ and $e^x - 1$ otherwise. Other features define other kernels. The feature width r need not equal d_k.

The numerator and denominator can each be accumulated independently of the query:

Normalized causal linear attention

With $\mathbf{S}_t \in \mathbb{R}^{d_v \times r}$ and $\mathbf{z}_t \in \mathbb{R}^r$, initialize both at zero and compute

$$\begin{aligned} \mathbf{S}_t &= \mathbf{S}_{t-1} + \mathbf{v}_t \phi(\mathbf{k}_t)^{\mathsf{T}}, \\ \mathbf{z}_t &= \mathbf{z}_{t-1} + \phi(\mathbf{k}_t), \\ \mathbf{o}_t &= \frac{\mathbf{S}_t \phi(\mathbf{q}_t)}{\mathbf{z}_t^{\mathsf{T}} \phi(\mathbf{q}_t)} = \frac{\sum_{j \le t} \kappa(\mathbf{q}_t, \mathbf{k}_j) \mathbf{v}_j}{\sum_{j \le t} \kappa(\mathbf{q}_t, \mathbf{k}_j)}. \end{aligned}$$

The denominator must be positive. The scalar division applies to every output coordinate.

If all feature vectors can be zero, handle a zero denominator explicitly. Numerical implementations also need protection against tiny denominators; adding an ϵ changes exact normalization slightly. Strictly positive features avoid an exact zero for a nonempty prefix in exact arithmetic, but do not eliminate numerical issues.

In the preceding example, use the nonnegative keys as feature vectors and query feature $(1, 1)^{\mathsf{T}}$. Now $\mathbf{z}_2 = (1, 1)^{\mathsf{T}}$, so normalization divides the raw output by two: $\mathbf{o} = (1/2, 2)^{\mathsf{T}}$, the average of the two values. The negative entry in a *value* is fine; nonnegative similarities are what make the weights a probability distribution.

Why this reassociation is possible. For a full sequence, stack feature rows in $\mathbf{\Phi}_Q, \mathbf{\Phi}_K \in \mathbb{R}^{T \times r}$ and value rows in $\mathbf{V} \in \mathbb{R}^{T \times d_v}$. The unnormalized computation can be grouped as

$$(\mathbf{\Phi}_Q \mathbf{\Phi}_K^{\mathsf{T}}) \mathbf{V} = \mathbf{\Phi}_Q (\mathbf{\Phi}_K^{\mathsf{T}} \mathbf{V}).$$

The right-hand form builds an $r \times d_v$ matrix rather than a $T \times T$ matrix. For causal attention, use *prefix* memories, not the full-sequence memory, or future values would leak into earlier predictions.

You cannot move softmax through the matrix product

In general, $\text{softmax}(\mathbf{QK}^\top)\mathbf{V}$ cannot be rewritten as $\mathbf{Q}(\mathbf{K}^\top\mathbf{V})$. The softmax kernel $\exp(\mathbf{q}^\top\mathbf{k}/\sqrt{d_k})$ must be replaced or approximated by a separable feature kernel to use this trick. The recurrent and parallel forms above are exactly equivalent for the *chosen feature kernel*, not automatically for ordinary softmax attention.

6.2 Recurrent Formulation and Complexity

"Linear" refers to how computation grows with sequence length when feature widths are fixed. It can also describe the dependence on the previous memory state. It does *not* mean the whole neural network is linear: projections, feature maps, gates, and surrounding FFNs can be nonlinear functions of the input.

The cost, with dimensions included

One write and read costs $\mathcal{O}(d_v r)$, plus $\mathcal{O}(r)$ for the normalizer. Across T tokens this is $\mathcal{O}(T d_v r)$, excluding the learned projections and feature-map computation. Recurrent inference stores $d_v r + r$ state entries per head, independent of T. By contrast, a softmax head stores $T(d_k + d_v)$ cached key/value entries and reads them for each new query.

For $r = d_v = d_k = 64$, normalized linear attention keeps $4096 + 64 = 4160$ numbers per head. At $T = 8192$, a softmax head's KV cache holds $8192(64 + 64) = 1{,}048{,}576$ numbers. This is a state-size comparison, not a claim that the two models have equal quality, equal total memory, or equal runtime. At short lengths or very large r, a matrix state can even be the more expensive option.

Constant-size *inference state* also does not imply constant total training memory. Backpropagation needs intermediate information, or must recompute it. Specialized scans and chunked algorithms determine the practical trade-off between saved activations, recomputation, and hardware efficiency.

Forgetting: keep the useful part of the past

Pure addition never removes an association. A gated version can decay memory before writing:

$$\mathbf{S}_t = f_t \mathbf{S}_{t-1} + i_t \mathbf{v}_t \phi(\mathbf{k}_t)^\top, \quad \mathbf{z}_t = f_t \mathbf{z}_{t-1} + i_t \phi(\mathbf{k}_t).$$

Using the same gates in $\mathbf{z}_t$ preserves the matching normalization. With nonnegative gates, write j receives weight $i_j \prod_{\ell=j+1}^{t} f_\ell$ at time t. A constant $0 < f < 1$ gives exponential fading; input-dependent gates can make retention depend on the content. Signed keys or different normalizers lead to different models, as we will see for mLSTM.

More generally, a matrix memory can evolve as

$$\mathbf{S}_t = \mathbf{S}_{t-1}\mathbf{M}_t + \mathbf{B}_t, \qquad \mathbf{B}_t = \mathbf{v}_t \mathbf{k}_t^\top, \qquad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t.$$

Here $\mathbf{M}_t$ transforms key-space coordinates. Diagonal transitions scale columns independently and cost $\mathcal{O}(d_v r)$; a generic dense $r \times r$ transition costs $\mathcal{O}(d_v r^2)$. An elementwise matrix gate can also scale every memory entry separately without a dense matrix multiplication.

The delta rule: correct an association instead of piling it up

Suppose the same key is encountered with a new value. Additive memory simply adds another write. A delta update first asks what the memory already returns for that key and writes the error:

$$\begin{aligned}\widehat{\mathbf{v}}_t &= \mathbf{S}_{t-1}\mathbf{k}_t,\\ \mathbf{S}_t &= \mathbf{S}_{t-1} + \beta_t(\mathbf{v}_t - \widehat{\mathbf{v}}_t)\mathbf{k}_t^\mathsf{T}\\ &= \mathbf{S}_{t-1}(\mathbf{I} - \beta_t\mathbf{k}_t\mathbf{k}_t^\mathsf{T}) + \beta_t\mathbf{v}_t\mathbf{k}_t^\mathsf{T}.\end{aligned}$$

The transition is an identity minus a rank-one matrix. There is no need to construct a dense transition: compute the memory-vector product and the outer-product correction directly in $\mathcal{O}(d_v r)$ time, taking keys of width r here.

Changing one stored value

Take the earlier $\mathbf{S} = \left(\begin{smallmatrix}2 & -1\\ 1 & 3\end{smallmatrix}\right)$, the unit key $\mathbf{k} = (1,0)^\mathsf{T}$, a new value $(5,0)^\mathsf{T}$, and $\beta = 1$. The old prediction is $(2,1)^\mathsf{T}$, so the correction is

$$(\mathbf{v} - \mathbf{S}\mathbf{k})\mathbf{k}^\mathsf{T} = \begin{pmatrix}3 & 0\\ -1 & 0\end{pmatrix}, \qquad \mathbf{S}_{\text{new}} = \begin{pmatrix}5 & -1\\ 0 & 3\end{pmatrix}.$$

The first association is replaced and the orthogonal second key still retrieves its old value. Pure addition would have stored $(7,1)^\mathsf{T}$ under the first key instead.

For a unit key, the new readout on that same key is $(1-\beta_t)\widehat{\mathbf{v}}_t + \beta_t\mathbf{v}_t$. Without unit norm, the effective correction is multiplied by $\|\mathbf{k}_t\|^2$. Other queries change in proportion to $\mathbf{k}_t^\mathsf{T}\mathbf{q}$, so overlapping associations can still interfere. The delta rule improves selective updating; it does not create an unlimited exact dictionary.

A diagonal-plus-low-rank transition $\mathbf{M}_t = \operatorname{diag}(\mathbf{d}_t) + \mathbf{a}_t\mathbf{b}_t^\mathsf{T}$ generalizes this idea:

$$\mathbf{S}\mathbf{M}_t = \mathbf{S}\operatorname{diag}(\mathbf{d}_t) + (\mathbf{S}\mathbf{a}_t)\mathbf{b}_t^\mathsf{T}.$$

Both terms are cheap. DeltaNet is the case $\mathbf{d}_t = \mathbf{1}$, $\mathbf{a}_t = -\beta_t\mathbf{k}_t$, $\mathbf{b}_t = \mathbf{k}_t$.

A recurrence can still train in parallel

Sequential inference is natural: keep one state, consume one token. During training, however, the whole input sequence to a layer is available. If transitions and writes depend on these inputs rather than on the unknown preceding state, the state updates are affine maps that can be composed associatively.

For $F_a(\mathbf{S}) = \mathbf{S}\mathbf{M}_a + \mathbf{B}_a$ followed by F_b,

$$F_b(F_a(\mathbf{S})) = \mathbf{S}(\mathbf{M}_a\mathbf{M}_b) + (\mathbf{B}_a\mathbf{M}_b + \mathbf{B}_b).$$

Representing an interval by $(\mathbf{M}, \mathbf{B})$, its combine operation is

$$(\mathbf{M}_a, \mathbf{B}_a) \star (\mathbf{M}_b, \mathbf{B}_b) = (\mathbf{M}_a\mathbf{M}_b,\ \mathbf{B}_a\mathbf{M}_b + \mathbf{B}_b).$$

A parallel prefix scan computes all prefix maps in $\mathcal{O}(\log T)$ parallel depth and $\mathcal{O}(T)$ combines using an appropriate work-efficient algorithm. This counts combine operations, not scalar arithmetic or wall-clock time. Associativity permits regrouping; it does not permit swapping token order.

Associative does not automatically mean cheap

The *whole affine update* must have a valid associative composition; naming an associative operator for only one term is insufficient. Dense transition composition can be expensive, and rank-one update factors need not stay rank-one when multiplied. Diagonal/elementwise transitions and specialized chunk algorithms matter in practice. Standard LSTM gates depend on the preceding hidden state, so this input-known scan argument does not apply to an arbitrary LSTM unchanged.

Reading the family of related models

The long list of names becomes manageable by asking what is retained, what is written, and how it is read. The shared matrix-state viewpoint groups the models as follows; it does not make their equations interchangeable.

Models	What changes in the memory?
Linear attention	Sum outer products; optional kernel features and a matching normalizer change similarity and scale.
Gated RFA, RetNet	Gate or decay the old memory. Gated RFA also tracks a matching normalizer; RetNet uses decay in its retention mechanism.
GLA, DFW	Use structured gates: GLA scales key-space coordinates; DFW applies structured elementwise decay.
RWKV-6, HGRN-2	Use gated matrix memories, with distinct choices of write and readout. Their gates alone do not specify the full model.
DeltaNet, Gated DeltaNet	Correct the value currently retrieved by a key; Gated DeltaNet also decays the memory.
ABC, GSA	Maintain separate key and value memories and retrieve through attention over compressed slots; GSA adds gating.
S4, Mamba, Mamba-2	Connect the recurrence to state-space dynamics, with different transition structures and input dependence.
mLSTM	Combine input/forget gating with a matrix memory and a stabilized readout.

The central trade-off is now visible: full attention preserves a growing bank for query-dependent retrieval; matrix-memory models repeatedly compress the history into a fixed state. That compression makes long sequences cheaper to process, while placing more responsibility on the write, forget, and read rules. The next chapters explore two concrete ways to make those rules useful: Mamba and xLSTM.

7

MAMBA

Not written yet.

Introduce selective state-space models and relate their architecture, execution, and empirical evaluation to recurrent and attention-based models.

Development plan: Planned figure: selective state-space block. Separate continuous/discrete state equations, input-dependent selection, and implementation-level parallelism.

7.1 Mamba Introduction

7.1.1 Core Idea

To be developed.

7.1.2 State Space Models

To be developed.

7.1.3 Selective State Spaces

To be developed.

7.1.4 Computational Complexity

To be developed.

7.1.5 Comparison with Transformers

To be developed.

7.1.6 Relation to RNNs

To be developed.

7.1.7 Limitations

To be developed.

7.1.8 Applications

To be developed.

7.2 Mamba Architecture

To be developed.

7.3 Mamba Experimental Evaluation

7.3.1 Mamba Scaling Laws

To be developed.

7.3.2 Mamba Zero-Shot Performance

To be developed.

8

xLSTM

Not written yet.

Explain how exponential gating and scalar or matrix memory extend LSTM, with separate treatment of equations, architecture, and experiments.

Development plan: Planned formulas: sLSTM, stabilizer state, and mLSTM. Planned comparison: storage capacity, state tracking, and parallel training. Keep experimental claims tied to their benchmark settings.

8.1 xLSTM Introduction

8.1.1 Background: Classical LSTM

Inability to Revise Storage Decisions. To be developed.

Limited Storage Capacities. To be developed.

No Training Parallelization. To be developed.

8.1.2 Motivation for xLSTM

To be developed.

8.1.3 Core Ideas of xLSTM

1. **Exponential Gating.** To be developed.

2. **Matrix Memory.** To be developed.

3. **Residualized Recurrent Blocks.** To be developed.

4. **Parallelized Training.** To be developed.

8.1.4 xLSTM Architecture Variants

sLSTM. To be developed.

mLSTM. To be developed.

8.1.5 Computational Properties

To be developed.

8.1.6 Comparison with Transformers

To be developed.

8.1.7 Comparison with Mamba

To be developed.

8.1.8 Strengths of xLSTM

To be developed.

8.1.9 Limitations

To be developed.

8.1.10 Broader Significance

To be developed.

8.2 xLSTM: Mathematical and Technical Details

8.2.1 sLSTM: scalarLSTM

To be developed.

8.2.2 Stabilizer State

To be developed.

8.2.3 mLSTM: matrixLSTM

To be developed.

8.2.4 mLSTM vs. sLSTM

To be developed.

8.2.5 xLSTM Architecture

To be developed.

8.3 xLSTM Experiments

8.3.1 Formal Languages and State Tracking

To be developed.

8.3.2 Memory Capacity

To be developed.

8.3.3 Language and LLM

To be developed.

8.3.4 Length Extrapolation

To be developed.

8.3.5 Scaling Laws

To be developed.

8.3.6 Speed Comparisons

To be developed.

8.3.7 Vision-LSTM (ViL)

To be developed.

8.3.8 Bio-xLSTM

To be developed.

8.3.9 Robotic xLSTM: LRAM

To be developed.

8.3.10 xLSTM Distillation

To be developed.

9

TiRex

Suppose you have a few weeks of electricity measurements and want to predict tomorrow. Training a large neural network just for this household would be a lot of work, and a few weeks may not provide much training data. A pretrained forecasting model offers a different starting point: give it the observed history and ask it to continue the series.

TiRex applies this idea using xLSTM recurrent memory. The useful questions are: what does the model learn before it sees our series, what changes while it reads our history, and how do we tell whether its forecast is any good?

9.1 Introduction to TiRex

9.1.1 Motivation

A time series can combine a slowly changing level, recurring patterns, sudden events, and measurement noise. The same forecasting method may face hourly energy demand today and daily sales tomorrow. There may be too little data to fit a separate complex model for each case, and the process itself may change over time. That last problem is *nonstationarity*: yesterday's relationship need not remain tomorrow's relationship.

Let $y_{1:T}$ denote the observed history and H the forecast horizon. A point forecast gives $\widehat{y}_{T+h}$ for $h = 1, \ldots, H$. A probabilistic forecast also describes uncertainty, for example through predicted quantiles. A median alone cannot tell us whether tomorrow's demand is fairly predictable or highly uncertain.

9.1.2 Core Idea

TiRex is pretrained across time series and then conditions its prediction on a new series' history. Its temporal backbone uses sLSTM, the scalar-memory component of xLSTM. Think of the recurrent state as a learned working summary that is updated as the model reads the sequence.

The original forecasting pipeline normalizes each series, embeds consecutive patches, processes them through recurrent blocks, and predicts quantiles. For horizons spanning several output patches, future inputs are marked missing rather than replaced with predicted medians. During pretraining, *contiguous patch masking* (CPM) hides consecutive input patches to prepare the model for this situation. The information flow remains causal.[1]

A patch is simply a short consecutive chunk, such as several neighboring measurements. It is turned into one vector before sequence processing. If each patch contains p measurements, a

[1] These pipeline details supplement the lecture overview; see the TiRex paper.

history of length T becomes approximately T/p recurrent steps. Patching reduces the number of steps while letting the embedding describe a local shape within each chunk.

Figure planned: From observed history to an uncertain forecast

Draw observed measurements, a normalization/patching stage, three recurrent blocks with a carried state, and a quantile forecast. Mark the forecast origin with a vertical line. Future input patches are missing, not measured zeros. Show median and 10–90 percent bands; label all curves illustrative.

Figure 9.1: The context updates a recurrent state; the forecast head turns that state into future predictions.

What preprocessing is allowed?. At a forecast origin, normalization statistics must come from available observations. For a context mean μ and positive scale s, normalized values are $\widetilde{y}_t = (y_t - \mu)/s$, and predictions are returned to the original units by $\widehat{y} = \mu + s\widehat{\widetilde{y}}$. A zero-variance context needs a safe scale convention. Missing values need a mask: a missing electricity measurement is different from a measured consumption of zero.

9.1.3 State Tracking with sLSTM

Here is enough of sLSTM to follow the forecasting idea. For one memory coordinate, write the idealized, unscaled recurrence as

$$c_t = f_t c_{t-1} + i_t z_t, \qquad n_t = f_t n_{t-1} + i_t, \qquad h_t = o_t \frac{c_t}{n_t}.$$

The candidate z_t supplies new content, positive gates i_t, f_t control its weight and retained history, and n_t tracks the matching normalization. The output gate o_t controls the exposed state. Vector versions apply these operations coordinatewise. Gates and candidates can depend on both the current input and the previous hidden state. Exponential gates allow a wide range of write strengths; practical implementations use stabilizers to avoid overflow.

"Scalar memory" means one scalar cell state per coordinate, not one number for the whole network. Many coordinates and layers together provide a learned representation. Unlike a fixed moving average, the model can decide how to update based on what it already remembers.

A normalized memory update

Take $c_{t-1} = 0.8$, $n_{t-1} = 2$, $f_t = 0.5$, $i_t = 2$, and candidate $z_t = 0.9$. Then

$$c_t = 0.5(0.8) + 2(0.9) = 2.2, \qquad n_t = 0.5(2) + 2 = 3.$$

The normalized memory is $c_t/n_t \approx 0.733$. Equivalently, the old normalized value 0.4 has weight $1/3$, and the new candidate 0.9 has weight $2/3$. With $o_t = 0.6$, the exposed value is $h_t = 0.44$. These are internal features, not a forecast in electricity units.

9.1.4 Foundation Model Perspective

Pretraining learns shared parameters from many sequences. At use time, a new history changes the model's *state*, while those learned *parameters* stay fixed. That distinction explains zero-shot forecasting: no task-specific parameter updates are needed to obtain a prediction.

Zero-shot does not mean zero history

A pretrained musician can continue a rhythm after listening to a few bars. Listening changes their understanding of this performance without requiring them to relearn music. Similarly, the history gives a forecasting model evidence about this particular series. It still needed pretraining, and it can still fail on patterns that its learned representation handles poorly.

A model evaluated on unseen datasets gives stronger evidence of transfer than one evaluated on data included in its pretraining corpus. Generalization across domains is a goal to test, not a guarantee attached to the words "foundation model".

9.1.5 Computational Advantages

For fixed architecture and patch width, consuming P patches requires $\mathcal{O}(P)$ recurrent steps. The carried state has a fixed size independent of P. This is attractive when processing long sequences: earlier observations can affect the current state without keeping a separate key and value for every old position.

These statements concern the temporal backbone. Total memory also includes weights, inputs, outputs, and temporary buffers. Training needs saved activations or recomputation. Moreover, an API that renormalizes a moving context and reruns the model is not automatically equivalent to indefinitely reusing one recurrent state. Efficient streaming needs a compatible preprocessing and state-management design.

9.1.6 Comparison with Transformer Forecasting Models

For the comparison below, fix width, depth, and patch size. The attention column describes dense, cached causal attention; different forecasting Transformers may use other mechanisms.

	Dense causal attention	**Recurrent temporal backbone**
History representation	Separate past keys and values	Compressed carried state
History storage	$\mathcal{O}(P)$ KV entries	$\mathcal{O}(1)$ state in P
Next temporal step	Reads up to P past entries	Updates a fixed-size state
Total over P steps	$\mathcal{O}(P^2)$ attention work	$\mathcal{O}(P)$ recurrent updates
Main trade-off	Direct access to past entries	Compression may lose details

Transformers can learn evolving dynamics too. The architectural difference is how that information is represented and updated, not a blanket inability of attention models to track state. Likewise, a fixed-size recurrent state does not promise perfect recall for arbitrarily long histories. Actual speed depends on kernels, batching, model size, and the forecasting setup.

9.1.7 Why State Tracking Matters

Imagine a machine whose temperature is currently 60 degrees. That number alone does not reveal whether it is warming up, cooling down, or running steadily. A history such as $(40, 50, 60)$ suggests a different continuation from $(80, 70, 60)$. The current observation is the same, but the underlying situation differs.

A useful latent representation might encode level, trend, phase, and operating regime. It need not contain separately labeled "temperature trend" or "machine mode" neurons. The recurrent update learns features useful for prediction. In sLSTM, the previous hidden state also influences how the next update is computed, so two identical observations can lead to different updates after different histories.

9.1.8 Applications

This setup fits energy demand, sales, sensor monitoring, and other forecasting tasks where historical patterns carry useful information. A forecast can also support anomaly detection: compare an arriving observation with the distribution predicted before seeing it. An unusual residual is evidence of surprise, not proof of a particular fault. Task-specific decisions still need an appropriate rule and evaluation.

9.1.9 Broader Significance

TiRex shows how recurrent memory can support the same broad workflow as other pretrained sequence models: learn from many examples, then adapt predictions through context. Its role here is the forecasting application of xLSTM. The important connection is between reusable learned dynamics and a state that changes for each new series.

9.2 Evaluations and Benchmarks for TiRex

A forecast is useful only if it predicts measurements that were unavailable when it was made. Choose a forecast origin T, supply only permitted information, predict the next H values, and compare with the subsequently observed targets. Repeat at multiple origins or across multiple series. Randomly mixing earlier and later time points into train/test sets can make a forecasting task unrealistically easy.

The benchmark families discussed here include Chronos-ZS, GIFT-Eval, fev-bench, and the live-evaluation projects TS-Arena and Impermanent. They compare different collections of domains, horizons, metrics, and forecasting setups. The reported TiRex results support strong performance in the evaluated settings; a leaderboard position is a snapshot, not a property of the architecture. The useful part to understand is what each score measures.

Point forecasts: MASE. Mean absolute scaled error compares the forecast's absolute error with an in-sample naive scale. For seasonal lag $m < T$, define

$$a = \frac{1}{T-m}\sum_{t=m+1}^{T}|y_t - y_{t-m}|, \qquad \text{MASE} = \frac{\frac{1}{H}\sum_{h=1}^{H}|y_{T+h} - \widehat{y}_{T+h}|}{a}.$$

For a nonseasonal series, use $m = 1$. Smaller is better; MASE below one means the test error is smaller than the *in-sample* naive scale, not necessarily smaller than the naive method's error on the same future horizon. If $a = 0$, the metric needs an explicit convention.[2]

[2]Metric definition: Hyndman and Athanasopoulos, forecast accuracy.

A score with an interpretable scale

Use history $(2, 4, 3, 5)$ and $m = 1$. Its naive scale is $a = (2 + 1 + 2)/3 = 5/3$. If the next observations are $(6, 8)$ and our point forecast is $(5, 7)$, the forecast MAE is 1. Hence MASE $= 1/(5/3) = 0.6$. This compares errors on a common scale; it does not mean "60 percent accurate".

Probabilistic forecasts: quantile loss. A predicted q-quantile $\widehat{y}^{(q)}$ aims to place about a fraction q of outcomes below it. Its pinball loss is

$$\rho_q(y - \widehat{y}^{(q)}) = \begin{cases} q(y - \widehat{y}^{(q)}), & y \geq \widehat{y}^{(q)}, \\ (1 - q)(\widehat{y}^{(q)} - y), & y < \widehat{y}^{(q)}. \end{cases}$$

For $q = 0.9$, underprediction costs nine times as much per unit as overprediction. That asymmetry encourages the prediction to move toward the upper tail. At $q = 0.5$, pinball loss is half the absolute error.

A common weighted quantile loss convention averages normalized losses over a chosen quantile set $\mathcal{Q}$:

$$\text{WQL} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{2 \sum_{h=1}^{H} \rho_q(y_{T+h} - \widehat{y}^{(q)}_{T+h})}{\sum_{h=1}^{H} |y_{T+h}|}.$$

The factor two matches the absolute-error scale at the median. Here the formula is for one series and origin; benchmark aggregation across series may use additional scaling or weighting. A zero denominator also needs a defined treatment. Unlike context normalization, using realized targets in a scoring denominator is allowed: scoring happens *after* forecasting.

For $y = 8$ and a 90th-percentile prediction of 7, the pinball loss is 0.9. Predicting 9 gives 0.1. A model therefore cannot optimize an upper-quantile forecast by always predicting the median. A collection of quantiles assesses more of the predictive distribution than a single point estimate does.

Intervals and uncertainty. The range between predicted 10th and 90th percentiles is a nominal 80% *prediction interval* for an individual future value. It is not a confidence interval for a fitted parameter and does not guarantee that 80% of entire future paths lie inside the plotted band. On held-out data, check both coverage and width: extremely wide intervals may cover well while being unhelpful. Marginal quantiles alone do not specify all dependence between future time points.

Reading the evidence. Keep the horizon, context length, zero-shot data overlap, and aggregation rule alongside the score. An overall average can hide a poor result in one domain. A helpful ablation changes one ingredient, such as the memory block or masking strategy, while keeping the remaining setup as comparable as possible. That gives better evidence about the ingredient's contribution than comparing two systems that differ in every component.

9.3 Applications of TiRex

For household energy, start with an evenly indexed history, choose the next-day horizon, and forecast before that day begins. Convert predictions back to the original units, compare with a seasonal baseline, and evaluate across many days. A visually convincing day is a useful illustration, but repeated held-out forecasts tell us whether the method is reliable.

Reading an illustrative energy forecast

Suppose tomorrow's total consumption has predicted quantiles $q_{0.1} = 8$, $q_{0.5} = 10$, and $q_{0.9} = 14$ kWh. The median forecast is 10 kWh and the nominal central 80% interval is $[8, 14]$ kWh. If tomorrow's measurement is 12 kWh, its absolute error against the median is 2 kWh and it falls inside the interval. That one successful inclusion does not establish 80% coverage: calibration needs many held-out forecasts. These numbers illustrate how to read the output; they are not a measured TiRex result.

Retail forecasts can inform stock planning, while sensor forecasts can provide a reference for maintenance monitoring. Those are downstream uses of predicted values and uncertainty. Classification is a different output task: a forecasting backbone can supply features, but class labels require a classification procedure and its own evaluation; they do not emerge automatically from forecast quantiles.

Original TiRex versus TiRex-2. The original model focuses on univariate forecasting. TiRex-2 extends the setup to joint multivariate targets and conditioning on past-only or known-future covariates.[3] A known-future input is something available when the forecast is issued, such as a holiday calendar or a scheduled promotion. Tomorrow's *observed* weather is not known today; a weather forecast issued today is available, but itself uncertain. This distinction prevents future information from quietly entering the model.

[3]Variant scope: TiRex-2 overview and covariate definitions.

10

Attractor Networks

So far, recurrence mostly meant reading a sequence over time. Here, we often give the network one pattern and let its internal state evolve repeatedly. A noisy pattern can move toward a stable stored pattern, much as a ball rolls into a nearby valley.

An *attractor* is a state or set that nearby dynamics approach. We focus on fixed-point attractors: once reached, the state stays there. Its *basin of attraction* is the set of starting states that converge to it. Associative recall means using a partial or corrupted cue to reach a stored pattern. The result depends on the dynamics and basin; it need not be the stored pattern closest in Euclidean or Hamming distance.

Figure planned: A cue rolls into a memory basin

Draw a schematic energy landscape with two desired memory minima and one spurious minimum. Put noisy cues on slopes and show their update paths. Label horizontal position as state configuration, not physical space; state that this is an analogy for discrete dynamics.

Figure 10.1: An attractor's basin determines which cues reach it; extra minima can produce unwanted recall.

10.1 Backpropagation for attractor networks with continuous time

First let the state settle. Consider N units with state $\mathbf{x}(t) \in \mathbb{R}^N$. Let $\mathcal{A}$ mark input units and Ω output units; they may overlap. Put fixed external drives into $a_i = \alpha_i \mathbb{1}_{i\in\mathcal{A}}$. The continuous dynamics are

$$\dot{\mathbf{x}} = -\mathbf{x} + \sigma(\mathbf{W}\mathbf{x}) + \mathbf{a}, \qquad \mathbf{W} \in \mathbb{R}^{N\times N}.$$

Here w_{ij} connects sending unit j to receiving unit i, and σ acts coordinatewise. The leak term $-\mathbf{x}$ opposes the current activity, while recurrent input and the external drive replenish it. The inputs are *drives*, not clamped state values: an input unit still obeys the differential equation.

Assume the dynamics converge to an equilibrium $\mathbf{x}^*$ for the input under consideration. At that point,

$$\mathbf{x}^* = \sigma(\mathbf{W}\mathbf{x}^*) + \mathbf{a}.$$

For desired output values $\boldsymbol{\omega}$, define the task loss

$$\mathcal{L} = \tfrac{1}{2}\sum_{k\in\Omega}(x_k^* - \omega_k)^2, \qquad e_k = \mathbb{1}_{k\in\Omega}(x_k^* - \omega_k).$$

This supervised loss is not automatically an energy that decreases while the state settles. State dynamics and parameter optimization are two different processes: hold the weights fixed while finding the equilibrium, then use its loss to update them.

Why differentiating the equilibrium is enough. Changing a weight changes the settled state, which changes the loss. Instead of storing every settling step, differentiate the fixed-point equation. Define

$$\mathbf{u} = \mathbf{W}\mathbf{x}^*, \qquad \mathbf{D} = \operatorname{diag}(\sigma'(u_1), \dots, \sigma'(u_N)), \qquad \mathbf{J} = \mathbf{D}\mathbf{W}.$$

For a small weight change $\mathrm{d}\mathbf{W}$ with the drive held fixed,

$$\mathrm{d}\mathbf{x}^* = \mathbf{D}\big((\mathrm{d}\mathbf{W})\mathbf{x}^* + \mathbf{W}\,\mathrm{d}\mathbf{x}^*\big), \qquad (\mathbf{I} - \mathbf{J})\,\mathrm{d}\mathbf{x}^* = \mathbf{D}(\mathrm{d}\mathbf{W})\mathbf{x}^*.$$

The second term is the recurrent effect: the changed state feeds back through the same network. If $\mathbf{I} - \mathbf{J}$ is invertible, this linear system gives the local equilibrium sensitivity.

An equilibrium gradient through one adjoint solve

Solve

$$(\mathbf{I} - \mathbf{J})^{\mathsf{T}}\boldsymbol{\lambda} = \mathbf{e}.$$

Then the weight gradient and a gradient-descent step are

$$\nabla_{\mathbf{W}}\mathcal{L} = (\mathbf{D}\boldsymbol{\lambda})(\mathbf{x}^*)^{\mathsf{T}}, \qquad \mathbf{W} \leftarrow \mathbf{W} - \eta\nabla_{\mathbf{W}}\mathcal{L}.$$

In coordinates, $\frac{\partial \mathcal{L}}{\partial w_{ij}} = \lambda_i \sigma'(u_i) x_j^*$. Solve the linear system rather than explicitly forming its inverse.

To see the adjoint idea, insert the sensitivity into $\mathrm{d}\mathcal{L} = \mathbf{e}^{\mathsf{T}}\,\mathrm{d}\mathbf{x}^*$ and move the inverse to the error side. The vector $\boldsymbol{\lambda}$ collects how equilibrium feedback amplifies or redirects output error. The final outer product looks local, but obtaining $\boldsymbol{\lambda}$ requires information from the network, not just one synapse.

Connection to repeated backpropagation. If $\rho(\mathbf{J}) < 1$, where ρ is the largest eigenvalue magnitude, then

$$(\mathbf{I} - \mathbf{J})^{-1} = \mathbf{I} + \mathbf{J} + \mathbf{J}^2 + \cdots.$$

Each power adds another trip around the recurrent connections. Equivalently, the iteration $\boldsymbol{\lambda}^{(r+1)} = \mathbf{e} + \mathbf{J}^{\mathsf{T}}\boldsymbol{\lambda}^{(r)}$ converges to the adjoint solution under this condition. A compatible matrix norm below one is sufficient, but not necessary, for convergence of the series.

Three conditions that should not be mixed up

The implicit derivative needs a differentiable equilibrium branch and an invertible $\mathbf{I}-\mathbf{J}$. The geometric series requires $\rho(\mathbf{J}) < 1$. Local exponential stability of the continuous dynamics is ensured when all eigenvalues of $-\mathbf{I}+\mathbf{J}$ have negative real parts. The last condition does not imply the series condition: the scalar case $J = -2$ gives stable continuous dynamics but a divergent geometric series. Near a singular equilibrium, gradients can also become very large.

One recurrent unit, with a checkable gradient

For a simple algebra check, use the linear activation $\sigma(u) = u$, weight $w = 0.5$, drive $a = 0.4$, and target $\omega = 1$. Then

$$\dot{x} = -(1-w)x + a, \qquad x^* = \frac{a}{1-w} = 0.8, \qquad \mathcal{L} = \tfrac{1}{2}(0.8-1)^2 = 0.02.$$

The adjoint is $\lambda = (0.8-1)/(1-0.5) = -0.4$, giving

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}w} = \lambda x^* = -0.32.$$

Direct differentiation gives $(x^*-1)a/(1-w)^2 = -0.32$ too. A small gradient-descent step increases w, strengthening feedback and moving the equilibrium toward the target.

10.2 Hopfield networks

A classical Hopfield network stores patterns in symmetric recurrent connections. Its states are bipolar, $\mathbf{x} \in \{-1, +1\}^D$, with $w_{ij} = w_{ji}$ and $w_{ii} = 0$. Define the local field

$$h_i = \sum_j w_{ij} x_j - b_i.$$

We use the lecture's threshold convention: positive b_i makes the $+1$ state harder to activate. Positive weights favor equal signs, while negative weights favor opposite signs.

To recall a pattern, initialize the network with a cue and update one unit at a time. Set x_i to $\operatorname{sign}(h_i)$ when $h_i \neq 0$; when $h_i = 0$, keep its old state. Each update uses the most recent values of the other units. This is asynchronous updating.

Why the updates settle. The energy is

$$E(\mathbf{x}) = -\tfrac{1}{2}\mathbf{x}^\mathsf{T}\mathbf{W}\mathbf{x} + \mathbf{b}^\mathsf{T}\mathbf{x}.$$

The half prevents double-counting symmetric pairs. If only unit i changes, all other energy terms cancel:

$$\Delta E = E(\mathbf{x}^{\text{new}}) - E(\mathbf{x}^{\text{old}}) = -(x_i^{\text{new}} - x_i^{\text{old}})h_i \leq 0.$$

A nonzero-field flip changes the sign to agree with the field and strictly lowers energy. A zero field causes no change under our tie rule. Since there are only finitely many binary states, there can be only finitely many flips. With a schedule that continues visiting every unit, the network reaches a state stable against any single-unit update. This is a local minimum, possibly non-strict along ties, not necessarily the global minimum.

How Hebbian storage works. For P bipolar patterns $\boldsymbol{\xi}^{(1)}, \ldots, \boldsymbol{\xi}^{(P)}$, the lecture's normalization is

$$\mathbf{W} = \frac{1}{P}\sum_{\mu=1}^{P} \boldsymbol{\xi}^{(\mu)}(\boldsymbol{\xi}^{(\mu)})^\mathsf{T} - \mathbf{I}_D.$$

Every diagonal entry of the averaged outer products is one, so subtracting the identity removes self-connections. Matching signs strengthen a connection; opposite signs make it more negative. Another common convention divides the off-diagonal sum by D. With zero thresholds, a positive global rescaling leaves sign decisions unchanged; with thresholds, the relative scale matters.

Repairing one wrong bit

Store one pattern $\boldsymbol{\xi} = (1, 1, -1, -1)^\mathsf{T}$ with zero thresholds. Its matrix is

$$\mathbf{W} = \begin{pmatrix} 0 & 1 & -1 & -1 \\ 1 & 0 & -1 & -1 \\ -1 & -1 & 0 & 1 \\ -1 & -1 & 1 & 0 \end{pmatrix}.$$

Start from the corrupted cue $(1, -1, -1, -1)^\mathsf{T}$. Unit 2 sees $h_2 = 1 + 1 + 1 = 3$, so it flips from -1 to $+1$, recovering the stored pattern. The energy falls from 0 to -6, consistent with $\Delta E = -(1 - (-1))3 = -6$. Every remaining unit already agrees with its field.

The sum of outer products resembles Chapter 6, but the use is different. Linear attention reads a value from a matrix. Hopfield recall repeatedly feeds the state through the matrix and a sign nonlinearity until it settles.

Where recall can go wrong. Contributions from multiple patterns can interfere. Stored patterns need not all remain stable under heavy loading, and extra stable states called *spurious attractors* may appear. With zero thresholds, sign-reversed patterns also share the same energy. The cue's basin and update order determine the outcome; Hebbian storage is not an exact nearest-neighbor lookup.

Synchronous updates can cycle

The one-unit energy proof does not apply when all units change together. Take $\mathbf{W} = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$, $\mathbf{b} = 0$, and state $(1, -1)^\mathsf{T}$. A synchronous update produces $(-1, 1)^\mathsf{T}$, and the next update returns to $(1, -1)^\mathsf{T}$. Both have energy $+1$. An asynchronous update instead makes the signs agree and reaches energy -1. Symmetric weights alone do not prevent synchronous cycles.

10.3 Boltzmann machine

A Boltzmann machine uses related energy-based interactions, but updates units by sampling. Low-energy states should occur more often; higher-energy states remain possible. This makes the model a distribution over patterns rather than only a deterministic recall procedure.

We now switch explicitly to $\mathbf{x} \in \{0, 1\}^D$. Keep symmetric weights, no self-connections, and the threshold-sign convention:

$$E(\mathbf{x}) = -\sum_{i<j} w_{ij} x_i x_j + \sum_i b_i x_i, \qquad p_\tau(\mathbf{x}) = \frac{\exp(-E(\mathbf{x})/\tau)}{Z_\tau}, \quad Z_\tau = \sum_{\mathbf{z} \in \{0,1\}^D} \exp(-E(\mathbf{z})/\tau).$$

Here $\tau > 0$ is temperature, not sequence length. The partition function Z_τ normalizes probabilities by summing over all 2^D configurations. This is a softmax over negative state energies. "Energy" is a modeling score; no physical units or Boltzmann constant are needed.

From energy to a sigmoid update. Hold every unit except i fixed. Turning i on changes its energy by

$$E(x_i = 1, \mathbf{x}_{-i}) - E(x_i = 0, \mathbf{x}_{-i}) = -\sum_j w_{ij} x_j + b_i = -h_i.$$

The ratio between the on and off probabilities is $e^{h_i/\tau}$. Normalizing those two possibilities gives

$$p(x_i = 1 \mid \mathbf{x}_{-i}) = \text{sigmoid}(h_i/\tau).$$

Choose a unit and sample its new bit from this Bernoulli probability. This is single-site Gibbs sampling. With finite energies, positive temperature, and suitable updates visiting every unit, it has the Boltzmann distribution as its stationary distribution. Mixing may nevertheless be slow when modes are separated by high barriers.

A complete two-bit distribution

Let $w_{12} = 2$, $b_1 = b_2 = 0$, and $\tau = 1$. States 00, 10, and 01 have energy 0; state 11 has energy -2. Thus

$$Z = 3 + e^2, \qquad p(11) = \frac{e^2}{3 + e^2} \approx 0.711,$$

while each other state has probability about 0.096. Conditional on $x_2 = 1$, the chance that x_1 is on is sigmoid(2) ≈ 0.881. Conditional on $x_2 = 0$, it is $1/2$. A lower-energy pattern is favored, but is not the only state sampled.

What temperature changes. For any two states,

$$\frac{p_\tau(\mathbf{x})}{p_\tau(\mathbf{z})} = \exp\left(-\frac{E(\mathbf{x}) - E(\mathbf{z})}{\tau}\right).$$

High temperature flattens the distribution. As $\tau \to 0^+$, the *equilibrium distribution* concentrates on global energy minima. That limiting statement does not mean a finite sampling run finds a global minimum. At positive temperature, the chain keeps moving; individual samples do not converge to one fixed state. At zero temperature, greedy local updates can get stuck, just as in Hopfield recall. At exactly zero local field, the sigmoid limit remains $1/2$, so a deterministic tie convention is an additional choice.

A useful distinction. In Hopfield recall, updates move a particular cue downhill until it is stable. In Boltzmann sampling, randomness allows uphill moves and aims to explore states with the correct relative frequencies. A probability distribution reaching equilibrium and a state reaching an attractor are different meanings of "convergence".

10.3.1 Learning in Boltzmann Machines

Training should raise the probability of data patterns relative to alternatives. For fully observed examples, maximize the average log likelihood $\ell = \mathbb{E}_{\text{data}}[\log p_\tau(\mathbf{x})]$. Differentiating a log probability gives a useful general identity:

$$\frac{\partial \log p_\tau(\mathbf{x})}{\partial \theta} = -\frac{1}{\tau}\frac{\partial E(\mathbf{x})}{\partial \theta} + \frac{1}{\tau}\mathbb{E}_{\text{model}}\left[\frac{\partial E(\mathbf{x})}{\partial \theta}\right].$$

The first term changes the observed example's energy. The second comes from differentiating $\log Z_\tau$: changing a weight also changes the competing states' probabilities.

Data statistics minus model statistics

For an independently parameterized symmetric edge w_{ij} with $i < j$,

$$\frac{\partial \ell}{\partial w_{ij}} = \frac{1}{\tau}(\mathbb{E}_{\text{data}}[x_i x_j] - \mathbb{E}_{\text{model}}[x_i x_j]).$$

For the threshold b_i used here,

$$\frac{\partial \ell}{\partial b_i} = \frac{1}{\tau}(\mathbb{E}_{\text{model}}[x_i] - \mathbb{E}_{\text{data}}[x_i]).$$

Use gradient *ascent* for ℓ, or reverse the signs for a negative-log-likelihood loss. Update each edge once and mirror it to preserve symmetry.

If two units co-activate more in the data than in the model, increase their coupling. If a unit is active too often in the model, increase its threshold. In the two-bit example, if every data point is 11, then at $\tau = 1$ the weight gradient is $1 - 0.711 \approx 0.289$. Merely strengthening observed connections without the model term would not perform maximum-likelihood learning.

Visible and hidden units. With $\mathbf{x} = (\mathbf{v}, \mathbf{h})$, only visible units are observed. The data term becomes a conditional expectation over $p(\mathbf{h} \mid \mathbf{v}_{\text{data}})$ with visibles held fixed, often called the positive phase. The model term is an expectation over the free joint model, the negative phase. Exact enumeration is exponentially expensive in a general Boltzmann machine, so inference and sampling dominate the practical difficulty of learning.

10.3.2 Restricted Boltzmann Machines

An RBM removes all connections within the visible group and within the hidden group. Only visible-hidden edges remain. This bipartite restriction makes the units of one group conditionally independent when the other group is fixed.

For D_v visible and D_h hidden units, use $\mathbf{W} \in \mathbb{R}^{D_v \times D_h}$ and thresholds $\mathbf{b}, \mathbf{c}$. At temperature one,

$$E(\mathbf{v}, \mathbf{h}) = -\mathbf{v}^\top \mathbf{W}\mathbf{h} + \mathbf{b}^\top \mathbf{v} + \mathbf{c}^\top \mathbf{h},$$

so

$$p(h_j = 1 \mid \mathbf{v}) = \text{sigmoid}((\mathbf{W}^\top \mathbf{v})_j - c_j), \qquad p(v_i = 1 \mid \mathbf{h}) = \text{sigmoid}((\mathbf{W}\mathbf{h})_i - b_i).$$

All hidden units can be sampled together given the visibles, and vice versa. Marginally they need not be independent.

Figure planned: Which connections make inference easier?

Draw three small graphs: a general Boltzmann machine, a bipartite RBM, and a visible–hidden1–hidden2 DBM. Mark visible nodes as observed. In the RBM highlight parallel hidden updates conditioned on visibles; in the DBM show both neighboring layers influencing hidden1.

Figure 10.2: Removing within-layer edges enables conditional block updates, but depth still couples hidden-layer inference.

The likelihood gradient still has data and model phases. *Contrastive divergence* CD-k starts a chain at a data vector, alternates hidden/visible sampling k times, and uses the short-chain statistics in place of the equilibrium negative phase. This is an approximate, generally biased

likelihood update. Persistent chains instead continue across parameter updates. Neither a short reconstruction nor a low reconstruction error proves that samples follow the learned equilibrium distribution.[1]

10.3.3 Deep Boltzmann Machines

A DBM adds several hidden layers with undirected connections between neighboring layers and none within a layer. For two hidden layers, a temperature-one energy is

$$E = -\mathbf{v}^\top \mathbf{W}_1 \mathbf{h}^{(1)} - (\mathbf{h}^{(1)})^\top \mathbf{W}_2 \mathbf{h}^{(2)} + \mathbf{b}^\top \mathbf{v} + \mathbf{c}_1^\top \mathbf{h}^{(1)} + \mathbf{c}_2^\top \mathbf{h}^{(2)}.$$

A first-hidden-layer unit receives information from both the visible layer below and the second hidden layer above. Knowing the visibles alone therefore does not make all hidden units independent. Inference typically uses iterative approximations, such as mean-field updates: replace hidden bits by approximate activation probabilities and repeatedly update their mutual influence. Learning also needs model samples for its negative phase.[2]

Depth provides multiple levels of latent representation, but makes inference harder than one RBM conditional pass. A DBM is not simply a feed-forward stack evaluated once, and it is not the same directed/undirected architecture as a deep belief network. The common thread is still energy: learning shapes which configurations receive high probability, while inference decides how to estimate or sample the unobserved parts.

[1] Practical learning details: Hinton's guide to training RBMs.
[2] See Salakhutdinov and Hinton, Deep Boltzmann Machines.

11

Hidden Markov Models

Not written yet.

Develop latent-state sequence modelling, inference, learning, decoding, and the HMM variants used in the lecture.

Development plan: Planned example: a small hidden-state sequence. Separate forward/backward inference, Baum-Welch parameter learning, and Viterbi path decoding.

11.1 Hidden Markov Models in Bioinformatics

To be developed.

11.2 Hidden Markov Model Basics

To be developed.

11.3 Expectation Maximization for HMM: Baum-Welch Algorithm

To be developed.

11.4 Viterbi Algorithm

To be developed.

11.5 Input Output Hidden Markov Models

To be developed.

11.6 Factorial Hidden Markov Models

To be developed.

11.7 Memory Input Output Factorial Hidden Markov Models

To be developed.

11.8 Tricks of the Trade

To be developed.

11.9 Profile Hidden Markov Models

To be developed.